

# A complete discussion on fully reconfigurable, digital, scalable, graph and sparsity-aware near-memory accelerator for graph neural networks

SIDDHARTHA RAMAN SUNDARA RAMAN, LIZY JOHN, and JAYDEEP P. KULKARNI, The University of Texas At Austin, USA

This is an extension of the paper published in ACM TACO 2024

Graph neural networks (GNNs) have gained significant interest for applications such as citation network analysis and drug discovery due to their ability to apply machine learning techniques on graph-structured data. GNNs typically employ a two-stage execution pipeline consisting of combination and aggregation kernels. The combination stage performs data-intensive convolution operations with relatively regular memory access patterns, whereas the aggregation stage operates on sparse graph data with highly irregular accesses. These heterogeneous memory behaviors make conventional CPU- and GPU-based execution energy inefficient due to substantial data movement overheads.

Existing accelerators attempt to mitigate these challenges using specialized architectures and processing-in-memory (PIM) techniques. However, prior approaches often suffer from scalability limitations, area overheads, restricted parallelism, and energy inefficiencies associated with analog compute and dedicated accelerator structures.

This paper presents NEM-GNN, a scalable DAC/ADC-less processing-in-memory architecture for graph neural network acceleration. The proposed design introduces early compute termination mechanisms, pre-computation using reconfigurable system-on-chip components, and graph- and sparsity-aware near-memory aggregation using a compute-as-soon-as-ready (CAR) and broadcast-based execution model. Experimental results demonstrate that NEM-GNN achieves approximately 80–230× higher performance, 80–300× higher throughput, 850–1134× better energy efficiency, and 7–8× higher compute density compared to prior state-of-the-art approaches.

**Additional Key Words and Phrases:** Graph neural networks, L1-cache, Processing In Memory, Compute-as-soon-as-ready, Broadcast, early compute termination, precompute sparsity-aware, graph-aware

## 1 INTRODUCTION

Over the past decade, there has been widespread utilization of deep learning models such as convolutional neural networks (CNN), and recommendation networks in diverse fields like image and video processing. These models primarily operate within the realm of Euclidean data, wherein data inputs conform to a structured and precisely defined  $n$ -dimensional space. However, these well-established models exhibit inefficiency when confronted with domains like citation networks, molecular chemistry, and electric grids [9][8], characterized by non-Euclidean data structures such as graphs and manifold geometries (3D surface structures). In these cases, the data inputs deviate from structured paradigms, and the pursuit to encapsulate them within a strictly defined  $n$ -dimensional space leads to loss of valuable information. Consequently, Geometric Deep Learning (GDL) [2] has emerged as a solution tailored to the idiosyncrasies of non-Euclidean data relationships. One of the approaches geared towards processing graph-based data hinges on the utilization of Graph Neural Network (GNN) models. These models are adept at handling graph-based inputs, facilitating the classification of nodes in a graph into distinct categorical groups.

Graph Neural Networks (GNNs) employ two fundamental mechanisms: (i) A combination kernel, akin to the convolution process in CNNs, is harnessed to encode the information within nodes. (ii)

---

Authors' address: Siddhartha Raman Sundara Raman; Lizy John; Jaydeep P. Kulkarni, The University of Texas At Austin, USA

This is an extension of the paper published in ACM TACO 2024.

Aggregation kernels are then utilized to encode the information in edges, which helps to comprehend the relationship between graph nodes. Regarding computations, the operations associated with the combination kernel are notably data-intensive, exhibiting regular memory access patterns. On the other hand, operations linked to the aggregation kernel are characterized by sparsity and irregularity. The mixed data pattern is a major limiter behind using CPU/GPU for GNN compute [38]. Various accelerator designs have been proposed to tackle this particular concern.

These accelerators can broadly be classified into traditional Von-Neumann-based accelerators/processing-in/near memory designs. The proposed designs (both Von-Neumann/PIM) are dedicated domain-specific accelerators, that require periodic interaction with the host, resulting in energy overhead. The existing Von-Neumann-based accelerator designs like AWB-GCN[7], HyGCN[38] incur data movement cost for transferring data from the processor to memory. Since the combination kernel uses data-intensive operations, requiring periodic data movement costs, this architecture is energy inefficient. The state-of-the-art processing in/near-memory (PIM/PNM) architectures for solving traditional graph algorithms, and GNNs, like ReFLIP [8] exhibit reduction in data movement by enabling computations within memory [27]. These architectures use crossbar Resistive Random Access Memory (ReRAM) arrays, Digital-to-Analog converters (DAC), Analog-to-Digital converters (ADC). The problems specifically with ReFLIP are: (i) This is a **dedicated accelerator** requiring periodic host-accelerator interaction, leading to energy inefficiency. (ii) The presence of DACs/ADCs renders the architecture susceptible to process variations, causing a decline in **accuracy**. (iii) Exponential increase in storage requirement with increased bit-precision/resolution for GNN compute, incurring **scalability** challenges. (iv) Aggregation is performed only upon the completion of combination operations, lacking any overlap between them, limiting **performance** (v) Sparse in-memory aggregation leads to **compute density** challenges. The major contributions are:

- **Re-configurability:** SOC components like L1/L2 cache are reconfigured to realize NEM-GNN without requiring memory array modifications/dedicated accelerator arrangement, thereby improving energy of the design.
- **DAC/ADC-less:** Bit-serial PIM architectures for combination (NEM-C1, NEM-C2, NEM-C3) with early termination of compute, pre-compute strategies without DAC/ADC, achieving highly accurate compute.
- **Scalability:** NEM-GNN is scalable to high bit-precision compute without requiring exponential storage for the compute array.
- **Graph aware:** Near-memory aggregation optimized specifically with "Compute-as-soon-as-ready" (CAR), "broadcast" approaches to hide aggregation latency, by overlapping combination and aggregation completely.
- **Sparsity aware:** Leveraging **sparsity** in kernels to minimize the unnecessary compute and utilizing the high-throughput of the design to improve compute density further.
- NEM-GNN outperforms ReFLIP by  $\sim 80$ - $230\times$ ,  $\sim 80$ - $300\times$ ,  $\sim 850$ - $1134\times$  and  $\sim 7$ - $8\times$  in terms of performance, throughput, energy efficiency and compute density [31].

## 2 BACKGROUND

### 2.1 Graph neural networks

Graphs, characterized by nodes and edges, have been a traditional and crucial data structure to represent unstructured data across a diverse range of real-world applications. In the pursuit of facilitating classification and clustering of graph nodes, considerable research effort has been devoted to Graph Neural Networks (GNNs). These networks are designed to extract distinctive features from graph structures and to enable an understanding of relationships among nodes within a graph. A variety of GNN variants, including Graph Convolutional Networks (GCN), Graph

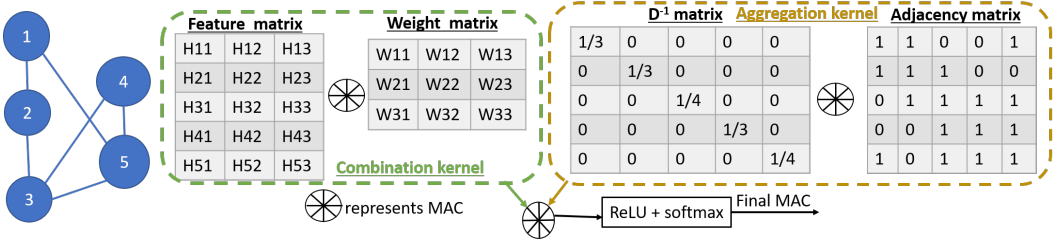


Fig. 1. Undirected, unweighted graph with 5 nodes and 6 edges passing through 1-layer GCN. Combination showing MAC between dense feature and weight matrices, aggregation showing MAC between sparse  $D^{-1}$ , adjacency matrices to generate final MAC before ReLU, softmax function

Attention Networks (GAT), and GraphSage [34], [35], are being extensively researched. These explorations are geared towards unraveling specific attributes of interest in various domains such as social network modeling, molecular chemistry, citation networks, etc. A Graph Neural Network (GNN) model is composed of multiple layers. In each GNN layer, every node from the input graph undergoes processing by two kernels: the combination and the aggregation kernel.

The function of the combination kernel is to convert the feature vectors (H), which characterize individual nodes in the graph, into an equivalent vector. This process entails the multiplication and accumulation (MAC) of the H values associated with each node in the graph using a weight matrix, akin to what is seen in a fully connected layer of a traditional neural network. The formulation of this kernel remains constant across all GNNs within the  $l^{\text{th}}$  layer and is expressed generically, for  $n^{\text{th}}$  node as:  $\mathbf{H}_{\text{comb}}[l][n] = \mathbf{H}_{\text{layer}}[l-1][n] * \mathbf{W}[l]$ . When  $l=1$ ,  $\mathbf{H}_{\text{layer}}[l-1]$  becomes same as input feature vector. For other layers, it is the final output of previous layer, similar to a traditional CNN. Across all nodes, feature vectors can be concatenated to form the feature matrix  $\mathbf{H}_{\text{comb}}[l] = \{\mathbf{H}_{\text{comb}}[l][n]\}$ .

The aggregation kernel combines attributes from neighboring nodes, represented by H, to gain insight into interactions with adjacent nodes and to mitigate data irregularities, by averaging across nodes [15]. This process varies across different GNN models. The aggregation mechanism for the  $l^{\text{th}}$  layer is denoted as  $\mathbf{H}_{\text{agg}}[l] = \mathbf{M} * \mathbf{H}_{\text{comb}}[l]$ , with the matrix M being contingent upon the specific GNN model in use. The aggregation mechanisms associated with different GNN models are:

For GCN,  $\mathbf{M} = \mathbf{D}^{-1} * \mathbf{A}$ , where D is the degree matrix normalized over node degree, typically represented as a diagonal matrix for easy matrix multiplication, and A is the adjacency matrix (Fig.1) across all neighbors. The adjacency matrix includes a self-loop, which results in the diagonal elements being set to 1. Similarly, the degree matrix is a diagonal matrix where  $D_{ii} = \sum A_i$ .

Example of GCN: In Fig. 1, a depiction of a graph with 5 nodes in an unweighted, undirected setup is shown. Each node contains 3 features, resulting in a feature matrix size of  $5*3$ . These features undergo a transformation using a weight matrix, akin to neural network convolutions. The weight matrix (combination kernel) is sized  $3*3$ , leading to a resultant combination matrix of  $5*3$ . Subsequently, aggregation follows, employing kernels consisting of the degree and adjacency matrix. For example, the feature vectors of node 1 (H11, H12, H13) transform based on the weight matrix. Node 1 has 3 adjacent nodes, accounting for self-looping. Consequently,  $D^{-1}_{11}$  becomes 1/3. The aggregated output for node 1 is a scaled sum of combination vectors of nodes 1, 2, and 5.

In the case of GraphSage,  $\mathbf{M} = \mathbf{D}_{\text{samp}}^{-1} * \mathbf{A}_{\text{samp}}$ , where  $\mathbf{D}_{\text{samp}}$  and  $\mathbf{A}_{\text{samp}}$  denote the sampled version of the degree and adjacency matrices respectively. In contrast to the approach of aggregating information from all neighbors, as seen in GCN, GraphSage exclusively employs a subset of chosen or pre-trained neighbors associated with a specific node for aggregation purposes. This selection process aims to ensure a consistent and predetermined count of neighbors for all nodes. Consequently, this strategy introduces a sense of relative preference among neighbors for aggregation.

For **GAT**,  $M = (\text{Attn})^*(A)$ , where Attn is the attention matrix. Unlike GCN and GraphSage, Attn relies on the combination vector of a node to determine the weight corresponding to each neighboring node. The Attn vector for a particular node is obtained by (i) Defining attention between  $i^{\text{th}}$  and  $j^{\text{th}}$  neighboring node as  $e^{a^*(\text{ReLU}(H_{\text{comb-}i} || H_{\text{comb-}j}))}$  where  $a$  is a trained vector and  $||$  indicates concatenation of combination vector of both the nodes (ii) Normalizing the attention across all the elements in a row of Attn matrix to refrain from numerical explosion.

ReLU is used as the activation function at the output of the aggregation layer to introduce non-linearity. Finally, softmax function is used for classification into different categories.

## 2.2 Processing in/near-memory

CPUs and GPUs have long served as the primary workhorses for various user applications, including GNNs. However, they face increased data movement costs due to their Von-Neumann architecture, requiring periodic data transfers between memory and computational units. Processing in Memory (PIM) aims to embed computations within memory, mitigating these costs. The choice of memory technology in PIM designs is crucial, with many GNN accelerators favoring ReRAM. However, ReRAM-based accelerators are often dedicated solely to GNNs, leading to increased dead silicon area when integrated into existing SOCs crowded with multiple accelerators. Understanding ReRAM characteristics is essential to grasp the drawbacks of these designs.

## 2.3 Resistive Random Access Memory (ReRAM) bitcell

This non-volatile memory element operates by varying resistance, unlike traditional charge-based memory like SRAM. It toggles between low-resistance (SET) and high-resistance (RESET) states, akin to '1' or '0' in SRAM/DRAM. The memory bitcell consists of 1T1R (1 Transistor 1 Resistor), wherein writing involves activating the corresponding word line (WL) and applying voltage to the bit line (BL) for SET/RESET. Reading relies on current flow, low for RESET and high for SET states. Despite their compactness, ReRAMs suffer from limited endurance, higher voltage requirements, latency, and susceptibility to process variations [27] [25][14][19]. Although, there are many drawbacks (which are overcome by SRAM bitcell), their compact nature is a driving factor [3][17][26].

## 2.4 Static Random Access Memory (SRAM) bitcell

SRAM stands out in cache/register file design for its ultra-low access latency (in the nanosecond range), surpassing other memory technologies [13]. Designs typically feature either a decoupled read/write port structure, seen in the 8-Transistor (8T) SRAM, or a shared read/write port structure, as seen in the 6-Transistor (6T) SRAM. The 8T SRAM employs WWL (Write word line) for writing and WBL (Write Bit Line) to store data in the storage node (S). During a read operation, RBL is precharged to a high voltage. If the S node holds '1', R2 is activated, discharging RBL through the R1-R2 stack. For a '0' in the bitcell, precharged RBL remains unchanged, aiding content distinction. The 8T SRAM performs efficient read-after-write (RAW) computations, unlike the typical 6T SRAM that necessitates write, precharge, and read commands, making RAW a 3-cycle operation. For 8T SRAMs, precharge is overlapped with the write command, thereby making RAW a 2-cycle operation, because RBL can be precharged, while a different row of bitcells is written using a combination of WWL/WBL. Also, these have lower write/read voltage/power as opposed to ReRAM, and resilience to process variations, thus giving performance, and power advantage [23][33].

## 2.5 Dynamic Random Access Memory (DRAM) bitcell

DRAM serves as the main memory storage due to its low cost and compact bit-cell design. The 1T1C (1-Transistor, 1-Capacitor) bitcell (Fig.2), utilizes a capacitor to store information. The write

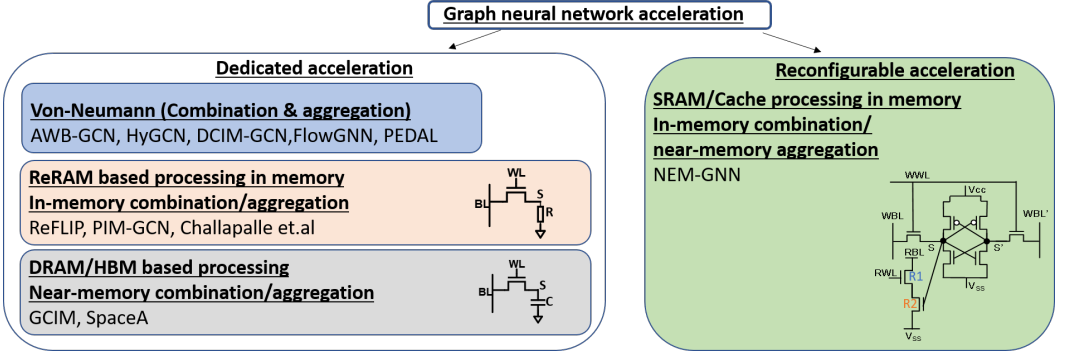


Fig. 2. **Landscape of Graph neural network based acceleration.** The prior works are predominantly dedicated accelerators requiring periodic host-accelerator interaction. These are further classified into Von-Neumann, ReRAM based PIM, DRAM/HBM based PIM. The proposed accelerator is not dedicated and reuses cache in CPUs to perform GCNs. The bitcells for PIM designs are also shown

operation involves activating WL while storing data onto the 'S' node by driving BL. Precharging the BL to half of the operating voltage precedes the read operation, where the voltage at BL changes based on the bitcell contents. The advantages of DRAM lie in its compactness, cost-effectiveness, and high capacity. Additionally, its capacity can be augmented through 3D stacking, enabling high-bandwidth memory (HBM)/high memory cube (HMC) designs. DRAM faces challenges such as periodic refresh due to the capacitor's dynamic nature, resulting in performance/energy overhead.

## 2.6 Related Work

The existing GNN accelerators are classified into Von-Neumann/PIM-based dedicated accelerators. Von-Neumann-based dedicated architectures like AWB-GCN[7], HyGCN [38] and DCIM-GCN[16], PEDAL [5], FlowGNN [29] make use of dedicated accelerator, specifically designed for GNN. The logic units are present outside the memory for performing combination and aggregation.

**AWB-GCN** introduces hardware-level enhancements aimed at addressing the challenge of workload imbalance, a consequence of the interaction between sparse input graphs and dense weight matrices. The proposed hybrid architecture employs distinct storage units for the combination and aggregation stages. This is complemented by specialized control logic that facilitates the seamless distribution of workloads across different processing engines. However, it's important to note that this strategy brings about an additional area overhead and offers only a modest assurance of optimal resource utilization. Moreover, the scalability of this solution is constrained by the number of processing elements. This is geared towards optimizing GCN predominantly.

**HyGCN** introduces an architectural approach and programming paradigm that harnesses both intra-vertex and inter-vertex parallelism during the combination and aggregation phases, thereby enhancing overall performance. Nevertheless, it's worth noting that despite these improvements, HyGCN lacks an inherent awareness of sparsity, leading to less efficient utilization of hardware resources and consequently resulting in an additional area overhead. Moreover, the inherent limitations of the Von-Neumann architecture become evident as graph sizes grow, contributing to the energy overhead caused by frequent data transfers between memory and computational units.

**DCIM-GCN** [16] employs memory as a storage component and supplements it with NOR gates/logic in proximity to the memory for near-memory combination. The aggregation, on the other hand, utilizes the Von-Neumann architecture and is specifically optimized for GCN.

**PEDAL** [5] introduces a power-efficient dataflow accelerator that optimizes dataflow based on the incoming graph's nature, enhancing efficiency and flexibility, by changing the order of

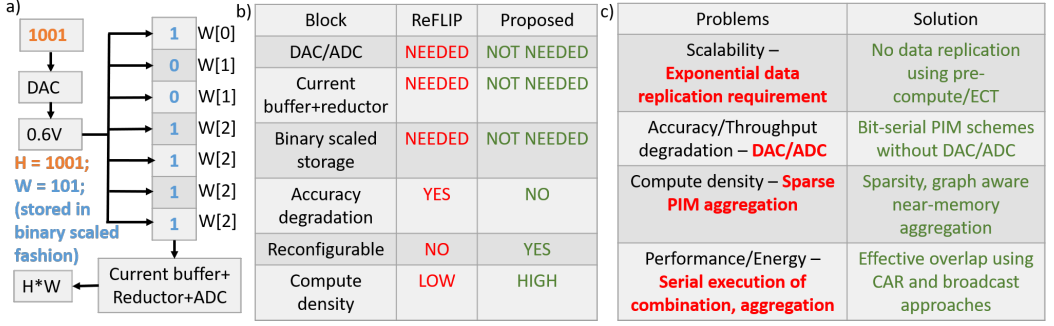


Fig. 3. a) ReRAM approaches (i) use DAC for incoming H conversion to an equivalent analog value (ii) store weights of GNN in binary scaled fashion (iii) utilize current buffer+reductor to perform current-based summation and ADC to generate  $H*W$  b) Qualitative comparison between ReRAM approaches and NEM-GNN c) A summary of the identified issues and the proposed solutions

execution between combination and aggregation. It employs a dedicated Von-Neumann architecture, necessitating periodic CPU-accelerator interaction.

**FlowGNN** [29] proposes a generic accelerator with parallelism ranging from node/edge, using multiple data queues, execution units along with scatter/gather mechanisms. The dataflow is general and can be applied to any graph based model like GCN, GAT, GraphSage. This architecture is realized on a FPGA, similar to other Von-Neumann accelerators, and suffer from similar drawbacks

PIM-based accelerators predominantly have utilized ReRAM/DRAM for in-memory processing. ReFLIP, PIM-GCN [39], Chappalle et.al [3] present Processing-in-Memory (PIM) accelerator that utilizes a crossbar ReRAM architecture. **ReFLIP** adopts a weight stationary approach for executing dot product operations, and it includes a peripheral DAC/ADC to perform analog compute. However, there are notable drawbacks associated with this approach, which are detailed in Sec.3.

**PIM-GCN** uses a similar approach to ReFLIP for performing MAC for combination, except that the aggregation is performed using a 2-step process with the first step being a content-address-memory (CAM) to identify neighbors, and the second step involves performing MAC. The proposed approach aims to schedule node computations in a way that the inter-node parallelism is maximal.

**Challapalle et.al** propose multiple engines with an architecture similar to PIM-GCN, except that the presence of separate engines potentially aids performance, at the cost of power/energy. However, the architecture is limited to performing GCN, cannot be extended to GAT/GraphSage

**DRAM/HBM** based near-memory accelerators make use of compute units closer to DRAM. **GCIM** [4] uses a novel data-aware mapping algorithm to efficiently utilize near-memory (3D-stacked HMC) MAC units. **SpaceA** [37] uses a near-memory CAM/MAC structure in processing engines to enable graph processing and perform workload balancing by mapping different sparse features to different banks, and is optimized for general graph processing and not for GCNs.

### 3 MOTIVATION

In this section, we motivate NEM-GNN by highlighting the issues in the existing PIM designs. The combination phase, predominantly consisting of convolution between weights and feature vectors, is naturally suitable for PIM compute. Furthermore, the dense weights are reused across various feature vectors, establishing the PIM-based combination as a weight-stationary approach. Aggregation is not amenable to PIM compute, as detailed later.

### 3.1 Issues in ReRAM-based PIM for combination

ReRAM device limitations are discussed in Sec.2.3 and also detailed in [1]. This section covers disadvantages, with specific reference to compute, as the existing ReRAM based accelerators follow the same strategy for performing in-memory dot-product compute, as that of ReFLIP. Fig.3a) illustrates dot product compute between  $H=1001$  and  $W=101$ . The weight bits are stored in a binary scaled fashion: the 0<sup>th</sup> bit is stored once, the 1<sup>st</sup> bit is replicated twice, and the 2<sup>nd</sup> bit is replicated four times. In the combination phase, the corresponding analog value for  $H$  is mapped onto the word line (WL), and the current flowing through the bit-line is aggregated using a current reductor. This is then fed into an Analog-to-Digital Converter (ADC) to derive the equivalent digital value. Firstly, the weights are stored in binary scaled fashion in memory. The incoming  $H$  value of 1001 is converted into an equivalent analog voltage of 0.6V by using DAC. This voltage then activates multiple bit-cells in the same column, by mapping  $H$  onto WL. The current summation across all ReRAM bitcells in a column (BL) is then performed by a combination of the current buffer and reductor. This is then converted into an equivalent voltage, and fed into ADC, which outputs an equivalent digital value of 101101. NEM-GNN overcomes the usage of analog blocks (summarized in Fig.3b). The disadvantages of the compute strategy are: (i) The presence of ADC and using ReRAM-based compute implies that the existing SOC components cannot be reconfigured to realize ReFLIP and need a dedicated accelerator arrangement. (ii) The binary scaled storage requirement requires that  $2^n-1$  rows are needed for storing an  $n$ -bit weight. This implies that the area of the memory array scales exponentially as the precision of weight increases, limiting scalability. (iii) ADC should be extremely precise, as an 8-bit weight multiplied by an 8-bit feature vector ( $H$ ) requires a 16-bit ADC. The process variations inherent to ADCs pose serious implications on the compute accuracy (Fig.3b). (iv) Throughput is restricted by the number of ADCs per bank (1 in the case of ReFLIP). Furthermore, the ADCs and current reductor affect the energy of the design with an additional area overhead arising from bulky ADCs. (v) Compute density, quantified as the ratio of operations performed within memory per bit-cell, serves as an indicator for gauging the efficiency of hardware resource utilization. In ReFLIP, for the dot product between an  $m$ -bit feature vector and an  $n$ -bit weight, the compute density equals  $(m*n)/(2^n)$ , which needs to be improved substantially. These issues are summarized in Fig.3c)

### 3.2 Issues in ReRAM-based PIM for aggregation

ReRAM-based PIM approaches perform both combination and aggregation in memory. However, aggregation is not really suitable for PIM computation, because of the following reasons: Firstly, realizing the exponential compute required for aggregation, like in GATs, is not suitable for PIM. Secondly, aggregation in memory can be achieved by 2 means: (i) The first option involves the usage of a separate memory array to store the resultant dot product from the combination phase (ii) The second option involves the reuse of the existing combination memory array for aggregation as well. The first option incurs additional area overhead for the associated memory array resulting in ineffective utilization of hardware resources, degrading the array density, and energy for compute. The second option improves the utilization of hardware resources at the cost of performance and additional control circuitry. Furthermore, this adds a degree of serialization between combination and aggregation, as aggregation can be initiated only after combination is complete. This causes a performance bottleneck with no overlap between combination and aggregation. Therefore both these approaches are inefficient in terms of performance, energy, and area. ReFLIP and PIM-GCN uses the latter, while Challapalle et.al use the former. Finally, the sparse aggregation compute, if performed in-memory degrades the compute density, as most PIM computes are insignificant.

### 3.3 Issues in DRAM-based near-memory compute for GNN

DRAMs, designed for cost efficiency and expanded storage, face constraints when integrating near-memory processing, leading to reduced storage capacity [28][24][18]. Given that DRAM is a single-chip package from manufacturers and directly integrated into existing SoC architectures, the fixed area allowance must accommodate additional near-DRAM logic. Notably, Samsung's near-DRAM chip for ML acceleration halved its storage capacity to include per-bank near-memory logic [11]. Consequently, traditional DRAM-hits turn into (DRAM+PIM)-misses, causing performance dips in storage-intensive workloads. This issue persists even in HBMs, with smaller storage capacities (e.g., recent HBM3 - 24GB vs. DDR4 - 256GB) and reduced storage density due to added bulky near-memory digital logic to DRAM bitcell arrays.

From GNNs' perspective, large datasets can push DRAM capacities to their limits. Storing basic node information (excluding edges/weights) with an 8-bit representation for features in PubMed/Reddit datasets alone demands 0.4GB/0.5GB. Preserving DRAM capacity becomes crucial to avoid the consequential power/energy expenses. Reduced DRAM capacity might limit concurrent CPU applications, and performance degradation in traditional CPU workload execution.

Samsung's demonstration focused on ML, optimizing for MAC operations. However, GNNs differ as aggregation navigates nodes, utilizing CAM operations, distinct from mere arithmetic operations. There are two approaches: i) Employ CAMs via traditional CPU, causing GNN execution degradation due to CPU-DRAM interaction, different from ML applications. ii) Including extra CAMs would further reduce storage capacity. Considering XNOR computation area vs. FP16 addition, this logic could trim DRAM by 10%. For instance, a reduced 3GB HBM-based PIM (originally 6GB, halved for ML in [11]) now reduced to 2.7GB, pushes DRAM limits, potentially causing misses, impacting performance/energy for datasets. Moreover, PIM's 3.5x ML performance gain would see a 10% reduction. Moreover, SpaceA augments HBM with specialized processing engines, expanding bit-width, incorporating CAMs, and load queues. This adaptation reduces DRAM capacity, impacting both traditional CPU tasks and GNN performance. GCIM adopts 3D-stacked HMC, encountering issues akin to SpaceA, facing bandwidth constraints leading to Micron halting its production in 2018. To summarize, Compute-near-DRAM approaches reduce DRAM capacity, causing performance and energy bottlenecks with increased misses, particularly for high-capacity workloads.

## 4 NEM-GNN'S SALIENT FEATURES

### 4.1 Reconfigurability

The requirement of dedicated accelerator arrangement in previous works is overcome by reusing the L1 cache inside the CPU core for performing in-memory compute with additional near-memory logic. Fig. 4a) shows a multi-core CPU design with each core integrating dedicated minimal near-memory logic for performing combination/aggregation. Fig. 4b) shows the per-core additional near-memory logic for L1 cache, assumed to be consisting of 2 banks for illustration. Shift and add are present at a granularity of 1 for every 8 columns (assuming weights for 8 bits) for every bank. 1 adder reduction tree/multiplier per bank along with buffer/control/counter shared across the entire L1 cache is used to realize NEM-GNN.

The datapath is split into combination and aggregation. For the combination data path, the compute array reuses the L1 cache to achieve PIM functionality with additional near-memory control logic for early compute termination and addition. For aggregation, we utilize a near-memory buffer for storing the adjacency matrix, a D generator for generating the corresponding degree(M) matrix, and UWC/WC engines for graph/sparsity-aware aggregation. If the underlying graph is unweighted, the UWC engine is used for aggregation, while the WC engine is used for weighted graphs. It is to be noted that these engines reuse the added near-memory logic like



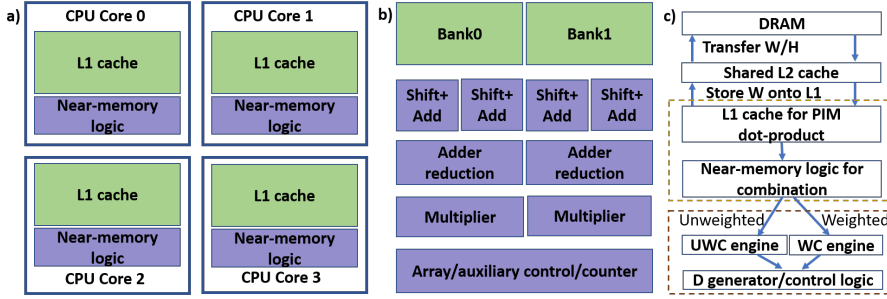


Fig. 4. a) NEM-GNN is realized by repurposing the L1 cache for in-memory compute, with minimal near-memory peripheral logic added to each CPU core. b) In an L1 cache, consisting of 2 banks, shift and add are present at a granularity of 1 per every 8 columns per bank, with 1 adder reduction/multiplier per bank, and other dedicated logic shared across the entire cache. c) DRAM is accessed to transfer weights/ feature vectors into shared L2 cache. L1 cache stores weights. Combination datapath involves PIM (L1 cache) dot-product with near-memory logic. The combination result goes via the UWC (Unweighted control)/WC (Weighted control) engine depending on the weighted nature of the underlying graph, followed by D-generator/control logic for aggregation. It is to be noted that UWC/WC engines use the added near-memory logic (multiplier/adder) and do not require dedicated hardware.

multiplier/adder and do not require dedicated logic for their functionality. There is no requirement for ADC/DAC/specific memory technology, enabling integration of NEM-GNN into a traditional CPU pipeline. For GNNs that do not fit on-chip, DRAM is accessed to fetch data into L1/L2. The access latency of DRAM is amortized by prefetching data, as PIM accesses are deterministic.

#### 4.2 Scalable L1 cache digital PIM compute without DAC/ADC - Architecture to circuit

Scalability is affected by exponential area requirement as the precision of weight increases. This is primarily because of the requirement for data replication across different banks. We propose 3 PIM approaches, which begin with  $n^2$  data replication (NEM-C1), and further reduce it to no replication requirement (NEM-C2/C3), as detailed in the next section.

Our proposed designs implement a digital bit-serial PIM methodology for combination that eliminates the necessity for DAC/ADC. This setup proves robust against process variations, ensuring accurate computations. With the employment of bit-serial computation devoid of DAC/ADC, the design's throughput is freed from ADC-related constraints. The bit-serial computation, in conjunction with strategies like early computation termination or pre-computation, facilitates achieving high performance/throughput for NEM-GNN designs. Moreover, the absence of data replication combined with bit-serial computation contributes to an enhanced compute density, elaborated in Section 5. Furthermore, given that i) ReRAMs suffer from scalability, compute density, performance/energy issues, ii) DRAMs suffer from reduced storage capacity, we propose L1-cache based PIM, that retains storage of DRAM, while offering improved performance/energy. From a memory organization standpoint, L1 cache consists of multiple tiles, with each tile consisting of multiple banks, leveraging tile and bank-level parallelism (as shown in Fig.5a). These designs use decoupled read/write port structure (8T SRAM), with the decoupled read port transistors marked as R1, R2 in Fig.5b). The read port transistors are re-purposed to perform dot product compute (bit-wise AND) for combination by mapping H bits onto RWL and storing weights in the bitcell. The compute can be described as follows: (i) RBL is precharged to  $V_{cc}$  before compute. (ii) Only when both S and RWL are '1', RBL discharges via the read-port transistors (R1/R2) implying that the computed value is 1. (iii) RBL remains at  $V_{cc}$  for other combinations of H and W, because one of

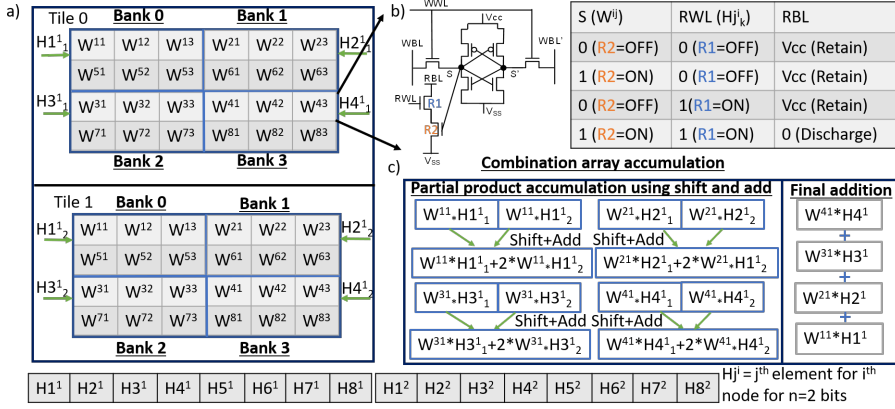


Fig. 5. a) Compute array organization for NEM-C1: 2 tiles with 4 banks in each tile, with bit-serial PIM performed between H mapped onto RWL and W replicated across both tiles is shown for illustration. 2-bit 8-element H and 1-bit  $8 \times 3$  weight matrix is shown with  $H_j^i$  indicating  $n^{\text{th}}$  bit of  $j^{\text{th}}$  element for  $i^{\text{th}}$  node. b) W is stored in 8T SRAM bitcell in L1 cache, and H is mapped onto RWL. RBL discharge is used as a measure of dot product between W and H. RBL is initially precharged to Vcc and discharges only when W and H are '1'. c) Shift and add across partial dot products from PIM compute, with the result of final addition written into a row of combination array

the read-port transistors are turned OFF. When  $W=0$ , R2 is turned OFF, as the S node is 0; When  $H=0$ , R1 is turned OFF, preventing discharge of RBL in either case. Using this PIM approach, we propose 3 different bit-serial scalable, performance and compute density optimized PIM approaches namely (a)  $n^2$  data replication, (b) no data replication with early compute termination (ECT), (c) pre-compute, which are detailed in NEM-C\* section.

### 4.3 Graph and sparsity-aware aggregation

We use a near-memory approach for aggregation, that enables performing exponential in-memory compute. The near-memory aggregation with CAR and broadcast approach helps achieve better performance in NEM-GNN, as aggregation latency is effectively hidden by computing as soon as the combination results are ready. This is done by effective broadcast of combination results, as soon as the combination outputs are available for aggregation.

Furthermore, the sparsity-aware approach helps alleviate insignificant computations, while leveraging PIM's high throughput for performing significant computations. Graph connectivity-aware aggregation helps optimize aggregation based on different graphs, enabling achieving high performance, without area overhead, for graphs that fundamentally require less computation for aggregation. For instance, unweighted graphs require less computation as compared to weighted graphs, because of no requirement on multiplying with weight of the graph edge for aggregation.

### 4.4 No impact on traditional CPU workloads

Since, NEM-GNN is realized reusing L1 cache, with additional near-memory logic, with no reduction in storage capacity, there is no impact on traditional CPU workloads. Firstly, there are no SRAM memory array level modifications to realize NEM-GNN, that could potentially impact the execution of traditional CPU workloads. Secondly, there is minimal near-memory logic that is added to realize NEM-GNN, achieved through effective reuse of hardware. For instance, the UWC/WC engines share the same adder, D-generator shares the multiplier with UWC/WC engines, and softmax shares

exponential compute, with aggregation for GAT. Thirdly, the additional near-memory logic does not intervene in normal CPU operation and does not add additional power overhead.

## 5 NEM-C\*: SCALABLE IN-MEMORY COMBINATION DESIGNS WITHOUT DAC/ADC

### 5.1 NEM-C1: Scalable bit-serial PIM with $n^2$ data replication

The exponential storage requirement ( $n$ -bit weight requiring  $2^n$  bitcells, independent of  $H$  bit-width) in ReFLIP is combatted by bit-serial PIM, with  $n$ -bit weight requiring  $m \cdot n$  bitcells for  $m$ -bit  $H$ , as detailed in the subsequent paragraph.

At the bank/tile level, the weights are kept stationary in the compute arrays, making the overall design a weight-stationary design. The weights are shown to be replicated for illustration of NEM-C1 design in Fig.5a) (while other designs do not require data replication). 8-element  $H$  vector each of 2-bits and 1-bit  $8 \times 3$  weight matrix is shown with  $H_j^i$  indicating  $n^{\text{th}}$  bit of  $j^{\text{th}}$  element for  $i^{\text{th}}$  node. The observation is that during dot product compute between  $H$  for each node (feature vector) and weight matrix to output combination vector, the same  $H$  element ( $H_1^1$ ) is used across a row of weights ( $W^{11}$ ,  $W^{12}$ ,  $W^{13}$ ). This re-use is used to map a row in the weight matrix onto a row in a compute bank, with  $H$  mapped onto the corresponding RWLs in a bit-serial fashion, enabling  $W^{xy}$  parallelism over  $y$ -dimension ( $H_1^1 \cdot W^{11}$  computed in parallel with  $H_1^1 \cdot W^{12}$ ). Inside a bank, eg. in bank0,  $H_1^1$  and  $H_5^1$  are mapped in successive cycles onto the RWL as a row is computed per bank, per cycle. The weight matrix is split across banks in a tile, enabling  $H_j^i$  parallelism over  $j$ -dimension ( $H_1^1 \cdot W^{11}$  computed in parallel with  $H_2^1 \cdot W^{21}$ ) and the weights are replicated across tiles for enabling  $H_j^i$  parallelism over  $n$ -dimension ( $H_1^1 \cdot W^{11}$  computed in parallel with  $H_1^2 \cdot W^{11}$ ). The replication of weights across multiple tiles ensures that the dot product between  $H_j^i$  and  $W^{xy}$  is computed using  $O(n^2)$  bitcells in a single cycle.

This is illustrated by using an example in Fig.5. If the different bits of  $H_j^i$  are mapped onto RWLs of the same banks in different tiles, then bank0 of tile0 computes  $W^{11} \cdot H_1^1$ , while bank0 of tile1 computes  $W^{11} \cdot H_1^2$ . These 2 partial dot products can be shifted and added together to generate the dot product  $W^{11} \cdot H_1^1$ . Similarly, dot products obtained across different banks and tiles are shifted and added to obtain  $W^{21} \cdot H_2^1$ ,  $W^{31} \cdot H_3^1$ , and  $W^{41} \cdot H_4^1$ . Finally, a full addition is performed to generate the MAC result for combination. Full adder accumulates dot product from SA into a row of combination array (Fig.5c).

The overall replication requirement is reduced to  $m \cdot n$  for dot product between  $m$ -bit  $H$  and  $n$ -bit  $W$ , resulting in  $O(n^2)$  data replication. Furthermore, the compute density for the L1-cache compute is improved from  $(m \cdot n)/(2^n)$  in ReFLIP to  $((m \cdot n)/(m \cdot n)=1)$  in NEM-C1, indicating effective utilization of hardware resources. Moreover, the lower overhead of SAs compared to DAC/ADC, and the lesser number of RWLs to be turned ON ( $n^2$  vs  $2^n$ ) leads to reduced energy requirement.

### 5.2 NEM-C2: Scalable bit-serial PIM with Early Compute Termination (ECT)

NEM-C2 aims to eliminate the need for weight replication in the compute array by utilizing previously computed values. This aims to enhance the efficiency of bit-serial computation in terms of area, energy, and compute density. For example, when storing  $3700 \times 10$  weights for a single layer of GCN in memory, where both feature vectors and weights are represented with 8-bit resolution, ReFLIP would demand about 1.2MB of storage, NEM-C1 would need around 0.2MB, and NEM-C2 would utilize only 4.6KB for storing these weights.

Similar to NEM-C1, multi-bit weights are stored in a single row of compute array, with elements of  $H$  mapped onto RWL in a bit-serial fashion, enabling  $W^{xy}$  parallelism over  $y$ -dimension. Various banks serve to accommodate distinct weight rows, thereby permitting  $H_j^i$  parallelism across the

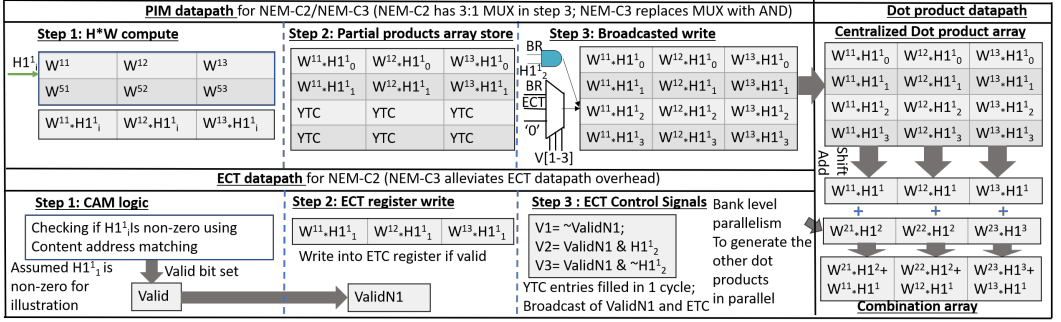


Fig. 6. **NEM-C2: Early compute termination (ECT)** occurs once one of the bit-serial H element bits is found to be 1, without data replication requirement. ECT data path checks for non-zero H bit in step 1 and writes the non-zero dot product into ECT register in step 2. In parallel, PIM datapath computes partial dot products in step 1 and subsequently stores them in the ECT register in step 2. This value is broadcasted to fill the other dot product entries in step 3 of PIM datapath using ECT control signals generated in step 3 of ECT datapath. 3:1 MUX for writing into the partial products array: BR/ECT/'0' indicate bank read (computed value from memory)/ read of ECT register/ zero value, respectively. Dot product datapath fills the combination array with the result of combination. **NEM-C3** eliminates ECT data path by pre-computing for H bit being 0/1, uses AND gate (marked in blue) instead of 3:1 MUX in step 3, while using PIM and dot product datapaths, similar to NEM-C2

j-dimension. Furthermore, parallelism across the i-dimension extends across tiles. There is no requirement for data replication across banks, unlike NEM-C1.

Fig.6 illustrates the interplay among distinct data paths: PIM, ECT, and the dot product path. In step 1 of PIM datapath, the "H\*W compute" stage generates the dot product between a row of weights and a bit of H element. Subsequently, in the second step of the PIM data path, the resulting dot product between  $H^{11}_1$  and ( $W^{11}$ ,  $W^{12}$ ,  $W^{13}$ ) is stored in the partial products array. It's notable that when one of the bits within the bit-serial H element is determined to be 1, it leads to the identification of both potential outcomes from multiplying a row by 0 and 1 (as multiplying by 0 would output 0). This knowledge can be leveraged to prematurely terminate the H\*W computation, thus saving energy. The ECT datapath provides assistance to terminate compute early, by identifying the position of '1' in a bit-stream of H mapped onto RWL. The Content-Address Matching (CAM) logic, allows for rapid searching based on content rather than memory addresses. This is utilized in step 1 of ECT data path in Fig.6 is used to identify whether H-bit is '1'. A valid bit is set if a non-zero element is found. The associated dot products between the non-zero  $H^{11}_1$  and ( $W^{11}$ ,  $W^{12}$ ,  $W^{13}$ ) are written into the ECT register in step 2, and the partial products array in parallel. Upon the completion of the ECT write operation, the remaining elements of the partial products array (denoted as YTC - Yet to Compute) are promptly filled by employing a single-cycle, broadcasted write using the data contained within the ECT register. This broadcast begins with the generation of ECT control signals in step 3 of the ECT data path, using the values of H elements and valid bit indication from CAM logic. Specifically, "BR" is chosen if the valid bit is '0', "ECT" is chosen if both the valid bit and the corresponding H bit are '1', and '0' is chosen if the valid bit is set while the corresponding H bit is '0'. The broadcast is done in step 3 of PIM datapath, using a 3 write-ported partial products array with 3 ports being bank read (when CAM logic has not detected a 1 in H element yet), ECT read (when ECT register is filled), and '0' (when H element bit is 0). Finally, the partial dot products are accumulated from the dot product array, onto the combination array, which is shown in the dot product datapath.

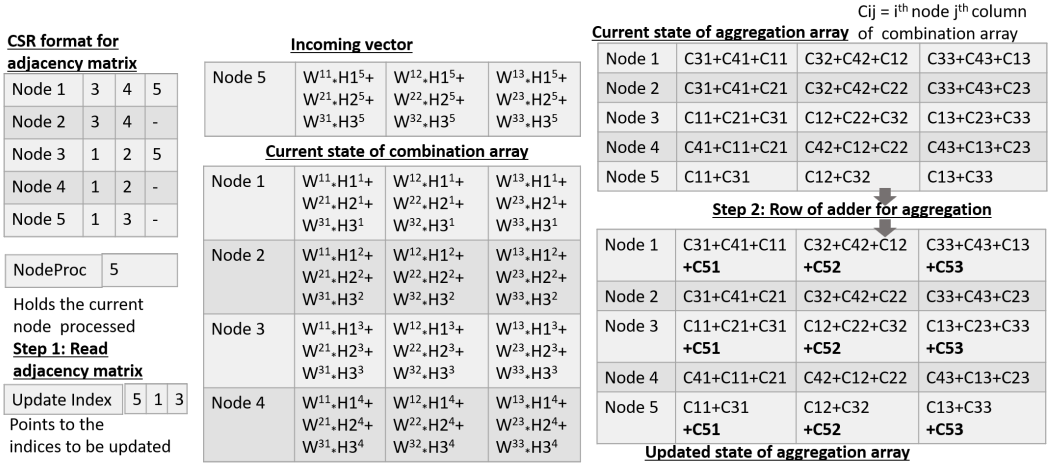


Fig. 7. Incoming graphs are mapped onto different engines based on graph-connectivity (graph-aware) and read-out of adjacency matrix (stored in Compressed Sparse Row Format) to eliminate unnecessary compute (sparsity-aware). UWC engine: Aggregation of unweighted graphs by reading the adjacency matrix and NodeProc register (indicating the node being processed by combination) to fill the update index register in step 1 and updating the aggregation array using adders in step 2. (ii) Node 5 (self-loop), Node 1 and 3 is aggregated with incoming combination vector for node 5, by "broadcasting" the vector (C51) to the aggregation array. The aggregation array is updated immediately with C51 once the combination result for a particular node is available, enabling "compute-as-soon-as-ready" approach

The broadcasted write turns off the compute array earlier, thereby improving the energy of the system. No weight replication for a bit-serial PIM approach improves the PIM compute density to  $(m \cdot n)/n = m$  along with reduced area requirements.

### 5.3 NEM-C3: Scalable bit-serial PIM with pre-compute

NEM-C2 introduces an ECT data path, which can be mitigated by leveraging the fact that all H bits are available from the storage array read. NEM-C3 tries to eliminate ECT overhead and enhance performance, by virtue of increased pre-computation.

Similar to NEM-C2, multi-bit weights are stored in a single compute array row, with H mapped onto RWL in a bit-serial fashion. Bank/tile-level parallelism for parallelism across j/i-dimension in  $H_j^i$  is leveraged, without requiring data replication (as shown in Fig.6).

During the computation of dot products, NEM-C2 awaits the first occurrence of '1' in the H element before terminating the compute. In contrast, in NEM-C3, the computation terminates even earlier by pre-computing dot products corresponding to both '1' and '0' H-bits. This approach obviates the need for the ECT datapath, while still utilizing the existing PIM and dot product datapaths to implement NEM-C3. The first 2 steps of the PIM data path remain the same as NEM-C2. However, in step 3, both the dot products are broadcasted to fill the partial-products array based on the H bit (shown as blue AND in Fig. 6), eliminating the overhead of ECT. This helps reduce 3 write ports (3:1 MUX) on the partial products array to a simple logical AND operation between the bank read and the bit for the row in partial products array ( $H1_2^1$  for 2<sup>nd</sup> row). This AND operation effectively signifies whether the array should be populated with 'zero' or the actual H value, depending on whether the H-bit mapped onto the RWL is '0' or '1'. The dot-product datapath itself remains unchanged from NEM-C3.

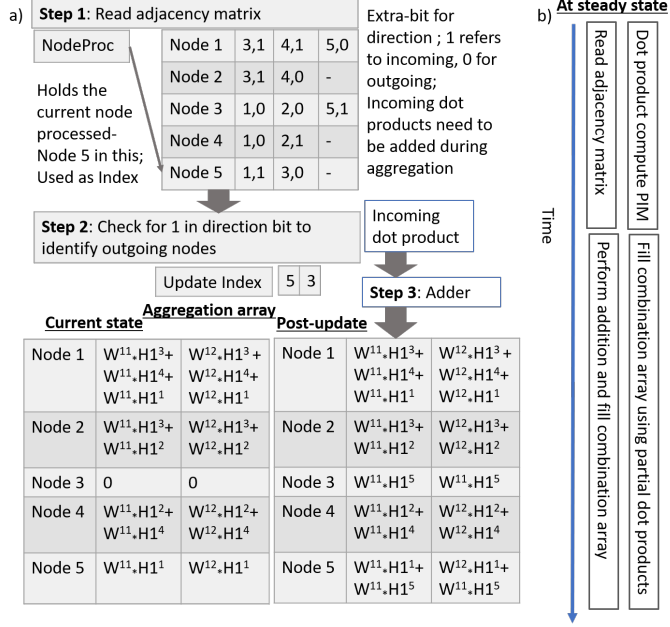


Fig. 8. a) **UWC engine: Aggregation for an unweighted, directed graph begins with reading the adjacency vector corresponding to Node Proc in Step 1, identifying outgoing nodes in step 2, and storing in Update Index register, using adders to aggregate the incoming combination vector onto the nodes in Update Index register in step 3. Each adjacency matrix element is of the form (i,j), where i/j represents the neighboring node/direction of interaction with the neighbor** b) **Timing diagram to show that dot product PIM for combination/read of the adjacency matrix of (n+1)<sup>th</sup>/(n)<sup>th</sup> node overlap; Filling of combination/aggregation array for (n+1)<sup>th</sup>/(n)<sup>th</sup> node overlap, hiding aggregation latency**

This results in overall area/power reduction, improved compute density due to ECT elimination, and reduction in write ports. Furthermore, the precompute strategy helps with achieving improved performance and energy, as the compute can be terminated earlier than NEM-C2.

## 6 GRAPH AND SPARSITY-AWARE NEAR MEMORY AGGREGATION

Aggregation involves a two-step procedure, with the first being the multiplication of the resultant combination output with the adjacency (A) matrix, and the second being the multiplication of the outcome from the first step with the D matrix (assuming GCN for simplicity of explanation). In the context of the first step, which is carried out in the WC/UWC engine, the computation is both graph-aware and sparsity-aware. The graph structures are intelligently optimized and distributed across different engines based on connectivity patterns, while the computation is made aware of matrix sparsity to mitigate unnecessary operations involving the A matrix. Additionally, we introduce two approaches: the "compute-as-soon-as-ready" (CAR) strategy, and "broadcast" technique. These techniques help overlap aggregation with combination, enabling resource reuse and concurrent processing. Moving on to the second step, this operation is executed within the D generator, and the process adopts a "sparsity-aware" approach. This technique strategically reduces the number of computations by accounting for the matrix's sparse characteristics.



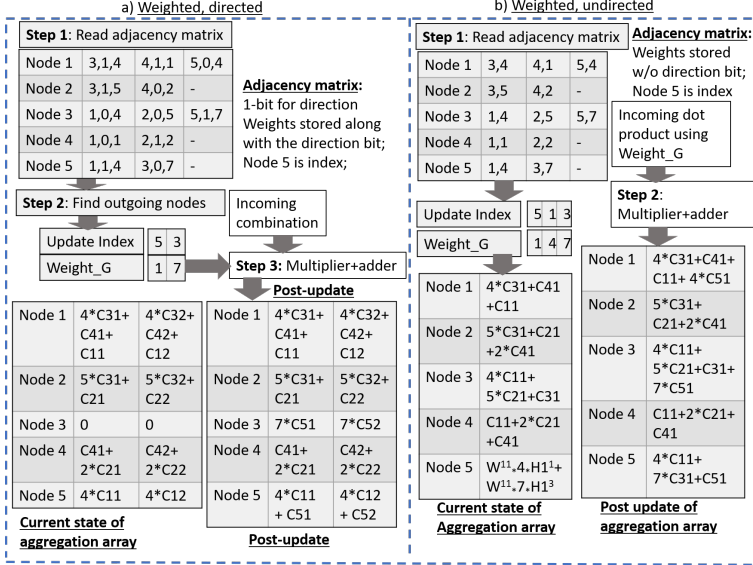


Fig. 9. a) Weighted, directed aggregation, with adjacency matrix storing the weights of graphs and the direction in the case of directed graphs. The direction is read out in step 1 to check for outgoing nodes in step 2 and aggregation with the incoming combination vector is achieved using near-memory multipliers and adders in step 3 b) Weighted, undirected aggregation follows the same datapath as the directed one, but without the notion of direction, making it a 2-step operation

## 6.1 Graph and sparsity-aware UWC engine for Unweighted graphs

While Graph Convolutional Networks (GCNs) find application across various graph types, their most frequent utilization occurs with unweighted graphs, notably in citation networks like Citeseer and Cora. We enhance the efficiency of these specific graphs by skillfully mapping them onto the Unweighted Control (UWC) engine. This mapping capitalizes on inherent graph patterns and reusability, effectively minimizing the hardware demands.

Several key challenges are evident from prior approaches: (i) In the context of ReRAM-PIM-based aggregation, the Processing-in-Memory (PIM) array computes partial dot products, necessitating an additional array (referred to as the "aggregation array") to store these partial dot products before they are collectively accumulated. This requires increased area. (ii) ReFLIP employs the same PIM array for both combination and aggregation, leading to a situation wherein aggregation only commences after the combination process concludes. (iii) The process of writing the resultant combination array onto PIM array before aggregation introduces an added energy cost. Contrasting this, NEM-GNN adopts a distinct strategy. It does away with the requirement for a PIM array dedicated to aggregation, instead using the "aggregation array" to initiate aggregation as soon as a fresh entry is introduced into the combination array. This facilitates the concurrent execution of aggregation and combination, resulting in improved performance, while eliminating the need for power and area overhead associated with PIM array write and read operations.

For aggregation in undirected, unweighted graphs, we begin by reading the adjacency vector (stored in compressed sparse row format (CSR)) for the node undergoing combination (indicated by the "NodeProc" register) in Fig.7. In step 1, reading the adjacency vector highlights nodes that share an adjacency with the specific node in focus. The identified adjacent nodes, along with the "NodeProc" information, are stored within the Update Index register to serve as aggregation candidates. The aggregation array, with indexing based on the Update Index register content,

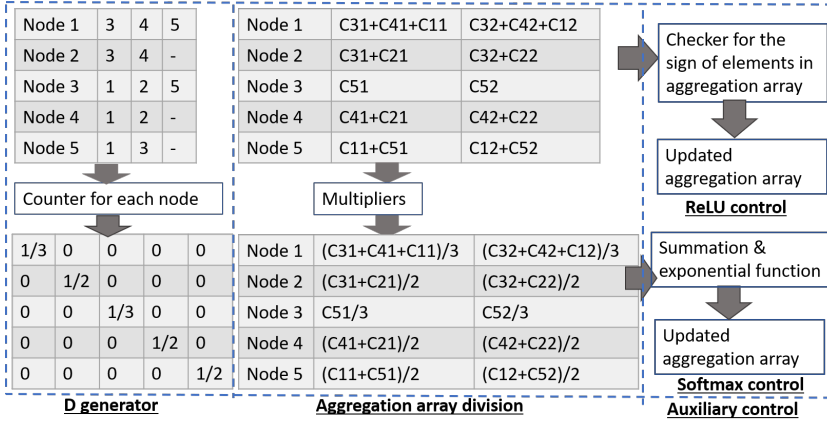


Fig. 10. **D-generator and control logic:** Degree matrix generator for generating  $D^{-1}$  using a sparsity-aware approach that (i) performs element-by-vector (instead of vector-by-vector) multiplication for every row, and (ii) reduces the number of computations/area by a factor of  $2n/n$ . Auxiliary control for ReLU and softmax is shown in the right-most figure.

undergoes immediate updates. This update involves the accumulation of the existing aggregation array values with the incoming combination vector through the utilization of a set of adders. This approach, characterized by simultaneous computation whenever data is ready, is known as the "Compute-as-soon-as-ready" strategy. The incoming combination vector is broadcasted to the aggregation candidates, allowing them to accumulate the combination vector concurrently. This mechanism facilitates "aggregation-vector" level parallelism. Fig. 7 shows that node 5 is the NodeProc by the combination data path. We identify that node 5 is connected to nodes 1 and 3 from the read-out of the adjacency matrix. Therefore, the "Update Index" register shows that nodes 5 (for self-loop), 1, and 3 are the aggregation candidates. Hence, the aggregation array entries of nodes 5, 1, and 3 are added with the incoming combination vector "C51, C52, C53" in step 2.

For the aggregation of directed, unweighted graphs, the adjacency matrix has an extra bit that shows the direction of interaction with the neighbors. '0'/'1' indicates an outgoing/incoming edge from/to a node. This is done so that only nodes having an outgoing edge from "NodeProc" are aggregated. Post-read-out of the adjacency matrix in step 1, CAM with the direction bit for the NodeProc is done to identify nodes that are to be aggregated in step 2 in Fig.8a). We utilize "CAR" and "broadcast" to improve performance in NEM-C3 as well. When NodeProc = 5, since node 5 has an outgoing edge to node 3 alone, the combination vector for node 5 is added with entries corresponding to nodes 5 and 3 of the aggregation array, using adders in step 3.

This approach eliminates redundant multiplications between the incoming combination vector and the adjacency vector when nodes are not adjacent. This refinement makes the design sensitive to sparsity, meaning the aggregation process no longer includes aggregation with a "0" weight for non-neighbor nodes. In Fig.8b), there's an evident overlap between aggregation and combination. This is achieved by concurrently reading the adjacency vector of the  $n^{\text{th}}$  node for aggregation while simultaneously calculating the dot product for the  $(n+1)^{\text{th}}$  node's combination. This overlap continues with the generation of partial products and the filling of the combination array for the  $(n+1)^{\text{th}}$  node, all while the aggregation process for the  $n^{\text{th}}$  node is underway. Due to the complete pipelining of both aggregation and combination, the latency introduced by aggregation is effectively hidden, resulting in improved performance.



|                                               |                                                                                                |                                                                                                                                                                                                                                                                                                                                 |                    |                                 |                                                                                                         |
|-----------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|---------------------------------|---------------------------------------------------------------------------------------------------------|
| Software support (Datasets/ testing networks) | Dataset - #nodes/edges/features/ Storage requirement / Operations                              | <b>Citeseer (CS)</b> – 3,327/9,104/3,703/40MB/2.3M<br><b>Cora (CA)</b> – 2,708/10,556/1,433/16MB/1.3M<br><b>Pubmed(PD)</b> - 19,717/88,648/500/41MB/18.6M<br><b>Nell (NL)</b> – 65,755/251,550/5,415/1.6GB/782M<br>CS1 – Weighted+directed Citeseer<br>CS2 – Weighted+undirected Citeseer<br>CS3 – Unweighted+directed Citeseer | Micro-architecture | Compute array                   | Tiles- 32; Banks – 256<br>Array Size – Upto 4KB                                                         |
|                                               | CS – Custom datasets with weights in graphs for testing WC engine<br>GCN/GAT/GraphSage network | 2-layer combination+aggregation; #hidden dimensions=128                                                                                                                                                                                                                                                                         |                    | Combination/ aggregation arrays | Combination – 128*32 flops<br>Aggregation – Banked flop array (256*128*32) entries                      |
|                                               |                                                                                                |                                                                                                                                                                                                                                                                                                                                 |                    | Area (mm <sup>2</sup> )         | Combination: SRAM – 4; Near-memory combi – 3, ECT – 1<br>Aggregation: UWC – 6, WC – 7.5; Aux logic – 2; |

Fig. 11. **Benchmarks: Datasets for GNNs, the number of nodes/edges/features in each of them, and the network used for GCN/GAT/GraphSage networks. Micro-architecture of NEM-GNN with the additional near-memory logic requiring 2% of AMD’s Zen3 CPU per-core area**

## 6.2 Graph and sparsity-aware WC engine for Weighted graphs

For weighted graphs, the adjacency matrix (A) is re-purposed to store the weight of interaction between 2 adjacent nodes. For compute of directed graphs, post read-out of A in step 1, we find the outgoing nodes of the NodeProc in step 2 to find the update index and weights (indicated as Weight\_G). Multipliers are used to perform the dot products between the incoming combination vector and the weight corresponding to the edges. Similar to the unweighted graphs, the incoming combination vector is "broadcasted" to accumulate onto the aggregation array corresponding to the update indices using adders in step 3. Fig. 9a) shows that there is an outgoing edge of weight=7 from node 5 to 3, which is multiplied with the incoming combination vector and aggregated onto node 3. For compute of undirected graphs, the adjacency matrix has additional bits (Fig. 9b) for storing the weight of the edge with no peripheral direction detection logic. The adjacency matrix is read in Step 1, to identify the adjacent nodes, which is followed by the "broadcasted" incoming combination vector using "CAR" approach in step 2. Apart from UWC engine’s advantages, WC engine consumes less power than ReFLIP, due to absence of power-hungry DACs/ADCs.

## 6.3 Sparsity-aware D generator and control logic

The key observation is that  $D^{-1}$  is sparse, and that multiplication of  $D^{-1}$  with aggregation array results in unnecessary computations. Therefore, we propose a **sparsity-aware approach** that consists of two main facets: (a) The approach involves exclusively storing the non-zero elements, specifically the diagonal elements, of the degree matrix ( $D^{-1}$ ). This curtails the required area by a factor of n. (b) Multiplying the aggregation array with  $D^{-1}$  is realized as element-by-vector multiplication for every row, reducing the number of computations by a factor of 2n, for  $D^{-1}$  of  $n*n$  and the aggregation arrays of  $n*n$ . This is because, when 2 arrays each of size  $n*n$  array are multiplied with each other, the total number of MAC operations would be  $\sim 2*n*n*n$ , assuming 1 operation each for multiplication and addition. However, the proposed approach necessitates only  $n*n$  multiplication operations without any accumulation operations. This is achieved by capitalizing on the characteristic of multiplication of a diagonal matrix ( $D^{-1}$ ) with an adjacency matrix. This involves multiplying each row in the adjacency matrix with the corresponding non-zero element in the same row of  $D^{-1}$ , thereby converting a vector-by-vector product to element-by-vector multiplication. Fig.10 shows that  $D^{-1}_{11}$  (1/3) (element) is multiplied with the first row of the aggregation array, instead of the first row of  $D^{-1}$  (like in ReFLIP). Furthermore, during compute, the degree matrix is generated in parallel with the first step of aggregation, followed by multipliers that perform multiplication between non-zero  $D^{-1}$  element and result of aggregation.

The advantages of this approach are that a) D matrix is re-used across different layers and b) D-generator is power-gated, once D-matrix is computed the 1<sup>st</sup> time. This helps achieve improved power/performance while reducing the number of unnecessary sparse computations. The auxiliary

**control logic** consists of (i) ReLU to identify the sign of elements and update the final aggregation array, (ii) softmax control logic employing exponential/summation to generate classification output.

## 7 ISA SUPPORT

NEM-GNN’s reconfigurability (L1/L2 cache used for compute/storage) helps re-purpose CPU instructions like load/store to read/write data onto the L1 cache for compute. Since the L1 cache can operate in 2 modes i.e. normal and compute mode, LCONF instruction configures L1 in compute mode by programming a special purpose register. Additional instructions for performing combination/aggregation are proposed. The instruction MACC Vx, VH, VW carries out an in-memory dot product, followed by near-memory accumulation. This process accomplishes combination between a row of weights (VW) and a feature vector (VH), ultimately storing the outcome in Vx. MACA Vagg, Va, Vx performs near-memory aggregation using the incoming combination vector (Vx) and adjacency vector stored in Va to store the aggregation result in Vagg.

## 8 EXPERIMENTAL METHODOLOGY

**Benchmarks:** A GNN model consisting of 2 graph convolutional layers with 128 hidden dimensions, similar to that of ReFLIP/AWB-GCN [8][7] is used as the underlying network to ensure a fair comparison between the proposed and existing designs. The quantization of GNN network weights, feature matrix, and weights of the input graphs (for weighted graphs) is done using PyTorch. The evaluated datasets and their properties are tabulated in Fig.11.

**Microarchitecture:** Compute array, repurposing 8T SRAM L1 cache is organized as 32 tiles/256 banks, matching L1 cache size of Intel Xeon E5-2680 v3. We integrate below mentioned NEM-GNN’s near-memory architecture to Intel Xeon E5-2680 v3’s architecture (wherever available) to ensure fair comparison, with a banking strategy, as mentioned below. The combination and aggregation arrays are implemented as registers, with the size of the combination array accommodating 128 W\*H dot products each of 32 bits. The aggregation array is organized as banks, with banks differentiated by node number (e.g. Bank0: 0 -255, Bank1: 256-511 nodes). The near-memory logic like shifters/adders are present at 1 per 8 columns, adder reduction/multiplier at 1 per bank.

**Performance/Power/Area evaluation:** Microarchitecture of **digital components** is specified using System Verilog, and synthesized with FreePDK 45nm [30] technology, operating voltage of 1V, clock latency of 2ns to identify the power and area tradeoffs. For the SRAM compute array and its peripheral circuits, power, and area are estimated using Cadence Virtuoso. **SRAM compute** energy is measured using an SRAM array of size 32\*256, with the sense amplifier offset voltage set to 100mV, RBL capacitance of 35fF, as measured from layout extraction with read/write latency of SRAM ~2ns, operating voltage of 1V. We use a custom performance simulator that accounts for the microarchitecture and the workload, to quantify the performance tradeoffs for combination and aggregation. For large-sized graphs not fitting on-chip, the overheads of (i) write latency of compute array is amortized by leveraging separate decoding circuits for write and read logic in 8T SRAM on the periphery, allowing write onto  $n^{\text{th}}$  row, when  $m^{\text{th}}$  row is computed, and (ii) data movement from DRAM to storage/compute array is amortized by data-prefetching. The data movement energy is 1pJ/bit (~ 800x addition energy) [12].

**Comparison Methodology:** NEM-GNN is compared against PyG-CPU, PyG-GPU (software optimized frameworks of PyG on CPU/GPU), AWB-GCN (non-PIM hardware accelerator) and ReFLIP (PIM hardware accelerator). **PyG-CPU** is PyG [6], a Python-based GCN-optimized library implementation of the GCN network on Intel Xeon E5-2680 v3, with 12 cores per socket and operating frequency of 2.5GHz, with L1 cache size of 64KB, L2 cache size of 256KB, L3 cache size of 2.5GB, along with DDR4 capacity of 256GB. Similarly, **PyG-GPU** is implemented on Nvidia

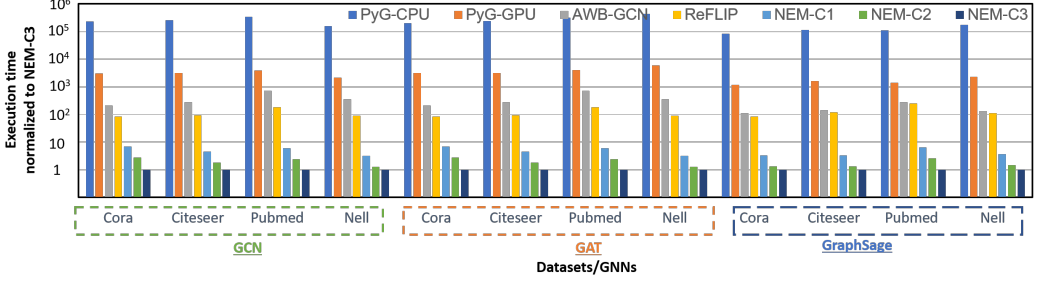


Fig. 12. **Performance comparison normalized to NEM-C3 for GCN, GAT and GraphSage.** UWC engine is used for aggregation, NEM-C1, NEM-C2, and NEM-C3 are used for combination.

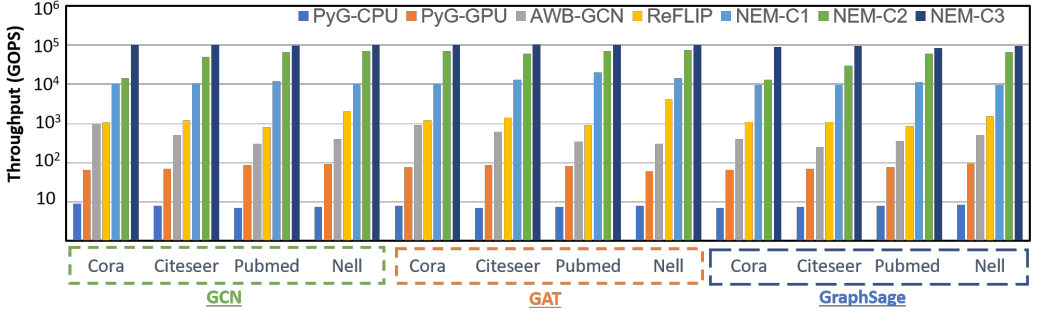


Fig. 13. **Throughput comparison measured in GOPS for GCN, GAT, and GraphSage.** UWC engine is used for aggregation, NEM-C1, NEM-C2, and NEM-C3 are used for combination.

Tesla v100, with 64 CUDA cores per streaming multiprocessor (SM) and an operating frequency of 1.5GHz, with 96KB L1 cache per SM, 6MB L2 cache and 16GB HBM2. **AWB-GCN**'s performance is obtained from its implementation on Intel D5005 FPGA with DRAM capacity of 32GB and 4096 PEs, frequency of 0.3GHz [7]. The performance of **ReFLIP** for combination is dependent on (i) the write latency of ReRAM (50.88ns [8]), (ii) the number of banks and (iii) the number of dot products computed per bank, (16384), limited by the number of DACs/ADCs per bank - 1 per bank in [8]. Aggregation needs to wait until combination completes and is further split into (i) A and (ii)  $D^{-1}$  matrix. Multiplication with A and  $D^{-1}$  needs to be done serially in a PIM array in GCN/GAT and multiplication with  $D^{-1}$  cannot be done in a PIM array for GraphSage because of the exponential function. The overall clock latency is limited by write latency of 50ns, operating voltage of 1V. DRAM prefetching is assumed for large-sized graphs, even in ReFLIP (for fairness). Power is obtained for (i) PyG-CPU using power-stat, (ii) PyG-GPU from Nvidia's system management interface (smi) (iii) Using the same configuration mentioned in [7] for AWB-GCN with rebalancing/distribution smoothing, and (iv) Using the power estimated in [8] for ReFLIP for DAC/ADC/ReRAM. The additional write costs onto PIM for aggregation, and data movement costs for large-sized graphs are also included. Energy is identified by multiplying the execution time with the power. For NEM-GNN, UWC/WC engine uses unweighted/weighted graphs for aggregation, and NEM-C1, NEM-C2, NEM-C3 are for combination.

## 9 RESULTS

**Performance** of NEM-GNN (Fig.12) is better for **GCN** than PyG-CPU because the cost of the increased data movement with increased graph size in CPUs is amortized by performing PIM. For PyG-GPU, the irregular memory accesses and limited on-chip memory restrict the performance of

the GPUs for larger workloads. For smaller workloads, the ineffective utilization of GPUs due to sparsity limits the performance of GPUs. This is reflected by the increased speedups, as high as  $\sim 10^4$  for Pubmed (large graph) and  $\sim 10^3$  for Cora (small graph). To summarize, concerning CPUs/GPUs, apart from in/near-memory compute, early-compute termination/pre-compute strategy, along with hiding latency using CAR and broadcast approaches for aggregation, helps achieve improved performance for NEM-GNN. For AWB-GCN, performance is limited by stalls arising from dynamic adjustment of workloads among 4096 PEs (by performing runtime optimization), and growth in the number of off-chip memory accesses. Furthermore, the multi-stage network (omega) coupled with the control logic necessary for distribution smoothing leads to performance bottlenecks, as it limits the parallelism obtainable across PEs. In NEM-GNN, PIM enables fast memory accesses with high parallelism across all columns/banks, without additional control logic for distribution smoothing, resulting in a speedup of  $10^4$ , compared to AWB-GCN in Pubmed. In ReFLIP, the major performance limiters are: (i) 1 DAC/ADC per bank limiting the number of dot products per bank to 1 (ii) serialization of combination and aggregation, (iii) high access latencies of ReRAM limiting the cycle time for a single dot product compute. The major advantages of NEM-GNN are (i) for combination, the dot product between a row of weights in a bank and a single H element mapped onto the bank, is computed in parallel (ii) the CAR scheme hides the latency of aggregation completely making the overall latency effectively the time taken to perform combination alone (iii) Lower SRAM access latencies, resulting in speedups of  $\sim 80x$ - $200x$  in almost all workloads. Particularly, NEM-GNN performs well when the ratio of the number of H to the number of nodes is low (observed in Pubmed), as the parallelism across the number of Hs/nodes is dependent on the number of banks/tiles. Among NEM-C\* designs, NEM-C1's performance is lower due to the limited parallelism across nodes, NEM-C2's performance is limited by the time to compute 1 node's combination array (as this is data dependent) and NEM-C3 combines the advantages of both NEM-C1 and NEM-C2 for better performance. For **GAT**, the number of computations is lower (assuming a fixed number of neighbors are predetermined), and NEM-GNN offers better performance ( $\sim 75$ - $140x$ ) than ReFLIP. For **GraphSage**, NEM-GNN achieves speedups of  $\sim 230x$ .

**Throughput** (Fig.12) captures the scalability of different designs. PyG-CPU is limited by the number of execution units capable of performing dot products. PyG-GPU is limited by the ability to handle sparse data. AWB-GCN is limited by the number of processing elements and workload rebalancing. ReFLIP's scalability is limited by the number of DAC/ADCs per bank, serial nature of combination and aggregation, and data movement onto the PIM array for initiating aggregation. NEM-GNN overcomes these by performing bit-serial computation, complete use of available throughput from the different columns in a bank (parallel dot product between a row of weights and incoming H element), pre-compute/ECT, parallel combination and aggregation, sparsity-aware compute leading to  $\sim 80$ - $300x$  times higher throughput than ReFLIP.

**Energy:** The overall power of NEM-GNN is divided into the power for (i) Combination control logic (ECT data path, dot product data path)/arrays (combination/dot-product array), (ii) UWC engine (adder)/arrays (adjacency matrix, aggregation array), (iii) auxiliary control logic, (iv) SRAM power across banks/tiles (v) Dot product array and multipliers in WC engine and (vi) ECT data path. The overall estimated power from (i) is  $\sim 3W$ , (ii) is  $\sim 6W$  and (iii) is  $\sim 1W$  and (iv) is  $\sim 2W$ . (v) and (vi) in NEM-C2 costs additional  $1.5W$  power (Fig.12)). Among NEM-GNN designs, NEM-C3 shows the least amount of energy (Fig.12) because the improved performance compensates for the power overhead. NEM-C2 has a slightly higher energy because of the lower performance and increased power from the additional ECT data path, as opposed to NEM-C3. Although NEM-C1 has the least power, the overall energy is higher because the decreased performance overcompensates

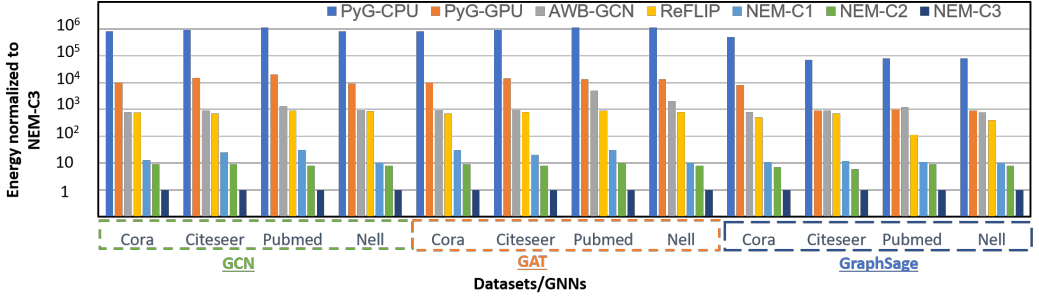


Fig. 14. Energy comparison for GCN, GAT and GraphSage. UWC engine is used for aggregation, NEM-C1, NEM-C2, and NEM-C3 are used for combination.

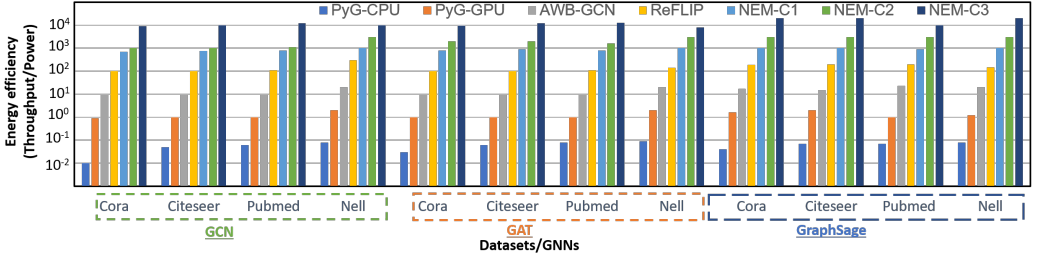


Fig. 15. Energy efficiency comparison for GCN, GAT and GraphSage. UWC engine is used for aggregation, NEM-C1, NEM-C2, and NEM-C3 are used for combination.

the lower power. In comparison to ReFLIP, NEM-GNN has the following advantages: (i) No power-hungry DAC/ADC requirements (ii) Lower write/read voltages for SRAM than ReRAM (iii) No additional write required to store back into the compute array post combination resulting in energy improvements of  $10^{2-3}$  in NEM-GNN (GCN). PyG-CPU, PyG-GPU, and AWB-GCN suffer from irregular memory accesses and frequent data movement, costing energy. Furthermore, there is no re-balancing of workloads like in AWB-GCN. In **GAT**, the energy improvement is higher because of the low power requirement in NEM-GNN, from lesser computations, along with the advantage from improved performance. In **GraphSage**, the power dissipated is higher (increased number of computations) with similar latency (latency of  $M$  matrix generation is completely hidden).

**Efficiency** is calculated by dividing the throughput by power (Fig.12). CPUs have limited on-chip memory and throughput, with the least efficiency. GPUs have higher throughput with many on-chip processing cores and improved on-chip memory capacity, indicating better energy efficiency as opposed to CPUs. AWB-GCN's decreased throughput is compensated by better energy, improving the efficiency over CPUs. ReFLIP performs PIM based compute with a higher degree of parallelism, improving throughput and energy. NEM-GNN has the highest amount of parallelism leading to increased throughput and decreased energy. Bit-serial PIM combination with pre-compute, and near-memory aggregation results in  $\sim 850$ - $1134\times$  improvement over ReFLIP.

**Utilization:** Fig.16a) tracks the resource utilization efficiency in terms of the compute density, which is equal to the number of computations (dot products) per unit area. The overall additional near-memory logic area required per CPU core is tabulated in Fig.11 and sums to  $0.47\text{mm}^2$ , which is 2% of AMDs Zen3-CPU area. UWC and WC engines share most of their data paths, thereby saving area. NEM-C1+UWC/WC engine has an area requirement of  $0.27\text{mm}^2$ . NEM-C2+UWC/WC engine has an additional  $0.2\text{mm}^2$  due to the presence of ECT data path, compared to NEM-C1 based design. NEM-C3 has negligible area increase due to the presence of extra AND gate, compared

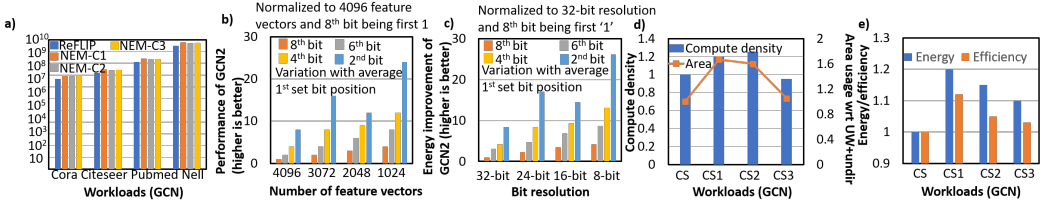


Fig. 16. a) Compute density comparison across PIM designs b) NEM-C2 performance variation with number of Hs c) NEM-C2 energy variation with bit resolution, average bit-position for first '1' d) Compute density, area for CS1, CS2 and CS3 e) Energy, efficiency for CS1, CS2, and CS3

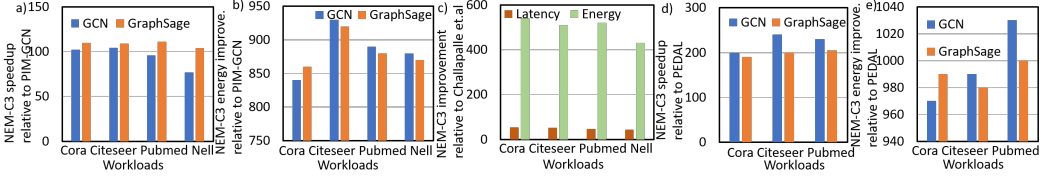


Fig. 17. a) Performance/b) energy improvement of NEM-C3 relative to PIM-GCN, c) Speedup/energy improvement relative to Challapalle et.al, d) Speedup, e) Energy of NEM-C3 relative to PEDAL

to NEM-C1 based design. The compute density is  $\sim 7\text{-}8\times$  that of ReFLIP, due to the elimination of bulky DACs/ADCs, no data replication, and sparsity-aware compute.

**Design space exploration:** The performance of NEM-C2 varies roughly linearly with (i) the average position of the first '1' of the incoming H vector, (ii) the number of Hs, as they determine the average time for combination per node and the required number of dot products (Fig.16b). For energy (Fig.16c), (i) lower bit-resolution causes lesser RBLs to discharge implying less power (for a fixed number of nodes/Hs) (ii) a decrease in the number of bits to obtain first '1' implying better performance. Both these factors combined show less energy for low resolution (e.g.8-bit over 16-bit) and decreased number of bits to obtain the first '1' (e.g.2-bit over 4-bit). The area (Fig.16d) required for evaluating CS1, CS2, and CS3 is more than CS, because of the additional multipliers for processing weighted edges, and the associated control logic for restricting the computation to outgoing edges. Compute density is the highest for evaluating CS2, because increase in area is compensated by increased number of computations. Fig.16e) shows that the energy (Power\*Performance) for CS1 is the highest, as there is a power increase from the multiplier and control logic (even when MAC for weighted edges is performed in a single clock cycle). The energy efficiency is the highest for CS1, as the energy increase is compensated by the increased number of computations.

## 10 ADDITIONAL COMPARISON RESULTS

In this section, we compare NEM-GNN's design, utilizing NEM-C3 for combination and UWC engine for aggregation, against other ReRAM-based PIM designs like PIM-GCN [39], Challapalle et al. [3], FlowGNN (state-of-the-art Von-Neumann accelerator), and PEDAL, focusing on GCN and GraphSage performance/energy metrics. As absolute throughput values aren't provided, a direct energy efficiency/throughput comparison isn't feasible, and GAT results aren't included.

PIM-GCN demonstrates speedups compared to PyG-CPU running on Intel Xeon E5-2680 v3 (the same hardware used in our simulations). Therefore, we employ the speedup/energy efficiency metrics reported in [39] for comparison. Our focus is limited to GCN and GraphSage for PIM-GCN, as the execution methodology for GAT isn't detailed in the article. While the disadvantages of ReFLIP are applicable to PIM-GCN, NEM-GNN designs achieve higher speedups. However, the achieved speedup is slightly greater compared to ReFLIP in smaller datasets like Cora/Citeseer,

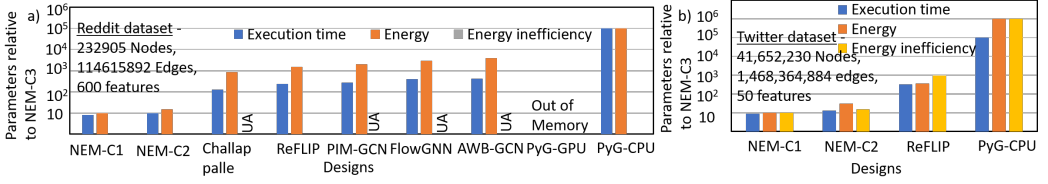


Fig. 18. a) Execution time/energy requirement/energy inefficiency of designs relative to NEM-C3 for a) Reddit dataset, b) Twitter dataset. UA means unavailable

mainly because PIM-GCN faces challenges in hiding additional latency for performing CAM to identify neighbors in the scheduling policy, whereas it performs better for larger datasets. This results in speedups of  $\sim 76x$ - $105x$ , as depicted in Fig. 17a). Similarly, in terms of energy efficiency, enhancements of  $\sim 840x$ - $940x$  are observed for GCN/GraphSage, as shown in Fig. 17b).

Challapalle et.al’s provided absolute performance/energy figures guide the comparison. Unlike ReFLIP, this architecture has distinct engines for traversal, combination, and aggregation, potentially enabling faster aggregation. However, NEM-GNN demonstrates speedups of  $\sim 45x$ - $53x$  and energy enhancements of  $\sim 430x$ - $570x$ , detailed in Fig. 17c). These gains stem mainly from high ReRAM write latency/power and challenges in completely concealing latency due to PIM compute. The combination engine must write results into the aggregation engine before in-memory computation, and all neighboring nodes’ combination vectors must be available before initiating aggregation for a node. Otherwise, frequent data transfers between main memory and ReRAM array incur power/latency costs. Moreover, lacking sparsity-aware compute, aside from CSR/CSC data representation, limits potential computational reductions, unlike NEM-GNN’s approach, leading to increased energy consumption in the UWC engine. It is important to note that PIM-GCN and Challapalle et.al’s modelling does not take into account the additional data movement cost associated with movement from host CPU to ReRAM-based memory array, and that would further improve speedup/energy associated with NEM-GNN.

We compare using latency/energy data from FlowGNN. While FlowGNN introduces the capability to compute graphs with edge embeddings, its performance remains nearly on par with previous accelerators for unweighted/undirected graphs when normalized to DSP usage. Hence, the energy efficiency improvements/speedups mirrored in NEM-GNN designs would resemble those seen in AWB-GCN, as depicted in Fig.12 and Fig.14 for GCN.

For PEDAL, we calculate latencies by multiplying workload cycles by clock frequency and determine energy by multiplying average power with latency. The highest performance across IP-AC, RW-AC, and RW-CA dataflows is reported. We observe  $\sim 200$ - $230x$ / $\sim 190$ - $205x$  performance improvement in GCN/GraphSage and  $\sim 960$ - $1030x$ / $\sim 980$ - $1000x$  energy improvement in GCN/GraphSage. These enhancements stem from factors like periodic host-accelerator interaction, constrained throughput from processing engines, and limited opportunities to conceal aggregation latency, owing to inherent Von-Neumann architecture constraints compared to PIM.

We compare prior works for Reddit dataset in terms of execution time, energy, energy inefficiency (wherever applicable). Similarly, comparison against Twitter dataset is performed against ReFLIP, PyG-CPU, as other designs have not reported their values. PyG-GPU has out of memory errors for both these datasets. Such large datasets show the scalability potential of NEM-GNN designs. The speedup reason for Von-Neumann architecture designs is the same as mentioned in Sec.9. Comparing NEM-GNN designs with other ReRAM based PIM designs, these are the following advantage: In large datasets like Reddit or Twitter, because we rely on the combination result to broadcast the data to the aggregation array (instead of aggregation process involving search for combination vectors corresponding to neighboring nodes to be aggregated for every node, like in ReFLIP), and



the possibility of combination vector getting broadcasted to more candidates increases. Since, the adjacency matrix is read in parallel to dot-product compute for combination, the broadcast approach can broadcast the combination vector to potentially larger number of aggregation candidates/nodes. Therefore, NEM-GNN is efficient for larger workloads, as well. Furthermore, this is not possible in other PIM designs, as combination vector results need to be written before initiating aggregation. These features enable improved performance/energy/efficiency of NEM-GNN, as shown in Fig.18.

NEM-GNN can integrate memory-compiled 8T SRAM arrays, improving compute performance and reducing production cycle time. Scaled to 65nm [36][22][32][10], our custom array is 1.5 times larger with 1.4x slower read/write latency and 1.6x higher power consumption than compiled memory, attributed to optimized layout reducing BL/WL capacitance. This boosts performance, power efficiency, and reduces footprint, resulting in 1.3x, 1.33x, and 1.35x performance benefits in GCNs, GATs, and GraphSage for NEM-GNN compared to the custom array. Energy efficiency improves up to 1.4x, 1.45x, and 1.47x for GCNs, GATs, and GraphSage, compared to the custom array NEM-GNN.

## 11 DISCUSSION

### 11.1 Adaptive Precision and Quantization Support

Another promising extension for NEM-GNN is the incorporation of adaptive precision execution and hierarchical quantization support. Modern GNN workloads often exhibit varying numerical sensitivity across layers, node types, and aggregation stages. Certain feature transformations require higher precision to preserve convergence accuracy, while aggregation operations can frequently tolerate aggressive quantization. Introducing dynamic precision scaling mechanisms can significantly improve energy efficiency and memory utilization without substantially affecting model quality.

The existing DAC/ADC-less digital execution model of NEM-GNN provides a strong foundation for flexible quantized computation. Since the architecture already avoids expensive analog conversion overheads, extending the datapath to support mixed-precision arithmetic can be achieved with relatively modest modifications. For example, low-precision aggregation units may be used for intermediate feature accumulation, while selective high-precision execution can be reserved for attention computation or normalization layers. Additionally, sparsity-aware quantization schemes may further reduce memory movement by compressing insignificant node features and edge weights.

A runtime-driven precision controller can dynamically adapt numerical formats based on workload behavior, graph sparsity, or layer sensitivity. Such techniques are especially beneficial for large-scale recommendation systems and graph transformers, where memory bandwidth often becomes the dominant bottleneck. Integrating adaptive quantization into NEM-GNN can therefore improve scalability while maintaining the architecture’s core energy-efficient near-memory design philosophy.

### 11.2 Runtime Scheduling and Graph Reordering Optimizations

Efficient runtime scheduling remains a major challenge for irregular graph workloads due to varying node degrees, load imbalance, and unpredictable memory access behavior. An important future extension for NEM-GNN is the development of hardware-software co-designed runtime schedulers that dynamically optimize graph execution order based on workload characteristics. Such runtime systems can improve accelerator utilization, reduce memory contention, and enhance parallelism across near-memory compute units.

The current compute-as-soon-as-ready execution strategy already provides an effective mechanism for latency hiding and asynchronous execution. Building upon this capability, future NEM-GNN designs may integrate graph reordering techniques such as degree-based clustering, locality-aware partitioning, or hot-node prioritization. These methods can improve memory locality and reduce repeated feature fetches during aggregation. Furthermore, dynamic work-stealing schedulers may redistribute computation across idle memory partitions to address load imbalance caused by skewed graph structures.

Another promising direction is integrating predictive scheduling mechanisms that leverage graph topology statistics or machine learning models to anticipate communication bottlenecks. Since graph workloads often exhibit recurring execution patterns, runtime systems can proactively optimize memory placement and aggregation ordering. Combining such runtime intelligence with the existing sparsity-aware NEM-GNN datapath can further improve throughput and scalability for large-scale industrial graph analytics applications.

### **11.3 Reliability, Thermal, and Power Delivery Optimizations**

Near-memory accelerators for graph processing introduce highly irregular current demand patterns due to sparse and bursty aggregation behavior. As NEM-GNN scales toward larger graph workloads and distributed memory systems, power delivery and thermal management become increasingly important research challenges. Simultaneous activation of multiple aggregation units can generate localized hotspots and transient voltage droop, potentially impacting timing reliability and long-term device lifetime.

Several architectural characteristics of NEM-GNN make it well suited for adaptive power-aware execution. The sparsity-aware aggregation framework already reduces unnecessary computation activity, indirectly lowering switching power and dynamic current demand. Additionally, the reconfigurable scheduling mechanisms may be extended to throttle computation based on thermal constraints or instantaneous power budgets. For example, aggregation tasks can be spatially distributed across memory partitions to avoid concentrated thermal hotspots while maintaining high throughput.

Future extensions may also incorporate power-aware graph scheduling and thermal balancing algorithms that monitor utilization across near-memory compute regions. Lightweight on-chip telemetry mechanisms can dynamically adjust aggregation intensity, communication frequency, or memory activation patterns to maintain stable operating conditions. Such reliability-aware enhancements are particularly important for large-scale datacenter deployments where sustained graph processing workloads can stress the power delivery network over extended execution periods.

### **11.4 Extending NEM-GNN to Broader Sparse Workloads**

Although NEM-GNN is primarily designed for graph neural network acceleration, many of its underlying architectural principles are broadly applicable to a wide range of sparse and irregular workloads. Applications such as sparse matrix-matrix multiplication (SpMM), sampled dense-dense matrix multiplication (SDDMM), PageRank, breadth-first search (BFS), sparse attention mechanisms, and integer linear programming (ILP) all exhibit irregular memory accesses, sparse computation patterns, and significant data movement overheads. Extending NEM-GNN beyond GNN inference therefore represents a promising direction for developing a general-purpose sparse near-memory accelerator.

A major advantage of NEM-GNN lies in its sparsity-aware execution model and compute-as-soon-as-ready scheduling framework. These mechanisms naturally align with sparse linear algebra kernels such as SpMM and SDDMM, where only a subset of matrix entries participate in useful computation. By leveraging the existing sparse aggregation datapaths, NEM-GNN can dynamically

skip zero-valued operands and avoid unnecessary memory accesses. Similarly, PageRank and BFS workloads rely heavily on sparse graph traversal and neighborhood propagation, making them suitable candidates for near-memory sparse aggregation and broadcast-based communication.

Several existing architectural components within NEM-GNN can already support these broader workloads with relatively modest modifications. The reconfigurable datapath and broadcast communication infrastructure may be reused for sparse feature dissemination, frontier propagation, and partial result accumulation. Furthermore, the early termination mechanisms originally designed for GNN aggregation can be adapted to sparse iterative algorithms where convergence occurs incrementally. For example, PageRank updates with negligible contribution may be pruned dynamically to reduce computation overhead and improve energy efficiency.

Another promising extension is supporting sparse attention workloads used in transformers and large language models. Sparse attention introduces irregular token dependencies and selective feature interactions that resemble graph aggregation behavior. NEM-GNN's sparsity-aware execution engine can potentially accelerate top-k attention, block-sparse attention, or locality-aware attention mechanisms by selectively processing only significant token interactions. Such extensions would substantially broaden the applicability of NEM-GNN toward emerging AI workloads beyond graph analytics.

In addition to machine learning applications, the architecture may also accelerate optimization-oriented sparse workloads such as integer linear programming (ILP). ILP solvers often involve sparse constraint matrices, branch-intensive execution, and irregular memory accesses that are poorly suited for conventional GPUs. The near-memory execution model of NEM-GNN can reduce data movement overhead during sparse constraint evaluation and matrix operations. Furthermore, its asynchronous compute [?] scheduling mechanisms can naturally support irregular branch-and-bound style execution patterns observed in optimization algorithms.

Future work may therefore explore transforming NEM-GNN into a unified sparse near-memory computing framework capable of supporting graph analytics, sparse AI models, optimization kernels, and scientific sparse linear algebra applications [21][20]. Such a generalized accelerator would address a growing class of workloads limited not by arithmetic throughput, but by irregular memory movement, sparse data dependencies, and inefficient data orchestration in traditional computing architectures.

## 12 CONCLUSION

We propose a scalable, reconfigurable, DAC, ADC-less, energy-efficient, high-performance GNN accelerator that reconfigures the L1 cache to realize PIM architecture for performing combination and a near memory approach for performing aggregation. For combination, bit serial PIM designs with early compute termination, and pre-compute are proposed to make the design scalable, without requiring data replication and DAC/ADC. For aggregation, graph, and sparsity-aware approaches leveraging the underlying graph connectivity, sparsity combined with "compute-as soon-as-ready" and "broadcast" approaches hide the aggregation latency to improve performance.

## REFERENCES

- [1] Pavan Kumar Reddy Boppidi, S. Siddhartha Raman, H. Renuka, and Souvik Kundu. 2020. Pt/Cu:ZnO/Nb:STO memristive dual port for cache memory applications. *AIP Conference Proceedings* 2265, 1 (11 2020), 030212. <https://doi.org/10.1063/5.0016597> arXiv:[https://pubs.aip.org/aip/acp/article-pdf/doi/10.1063/5.0016597/14105127/030212\\_1\\_online.pdf](https://pubs.aip.org/aip/acp/article-pdf/doi/10.1063/5.0016597/14105127/030212_1_online.pdf)
- [2] Michael M Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. 2017. Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine* 34, 4 (2017), 18–42.
- [3] Nagadastagiri Challapalle, Karthik Swaminathan, Nandhini Chandramoorthy, and Vijaykrishnan Narayanan. 2021. Crossbar based Processing in Memory Accelerator Architecture for Graph Convolutional Networks. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9. <https://doi.org/10.1109/ICCAD51958.2021.9643465>

- [4] Jiaxian Chen, Yiquan Lin, Kaoyi Sun, Jiexin Chen, Chenlin Ma, Rui Mao, and Yi Wang. 2022. GCIM: Toward Efficient Processing of Graph Convolutional Networks in 3D-Stacked Memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 3579–3590. <https://doi.org/10.1109/TCAD.2022.3198320>
- [5] Yuhan Chen, Alireza Khadem, Xin He, Nishil Talati, Tanvir Ahmed Khan, and Trevor Mudge. 2023. PEDAL: A Power Efficient GCN Accelerator with Multiple Dataflows. In *2023 Design, Automation & Test in Europe Conference Exhibition (DATE)*. 1–6. <https://doi.org/10.23919/DATe56975.2023.10137240>
- [6] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).
- [7] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, et al. 2020. AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 922–936.
- [8] Yu Huang, Long Zheng, Pengcheng Yao, Qinggang Wang, Xiaofei Liao, Hai Jin, and Jingling Xue. 2022. Accelerating Graph Convolutional Networks Using Crossbar-based Processing-In-Memory Architectures. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1029–1042.
- [9] Chanwoo Jeong, Sion Jang, Eunjeong Park, and Sungchul Choi. 2020. A context-aware citation recommendation model with BERT and graph convolutional networks. *Scientometrics* 124, 3 (2020), 1907–1922.
- [10] Jaydeep P. Kulkarni, Siddhartha Raman Sundara Raman, Shanshan Xie, and Chieh-Pu Lo. 2025. Unconventional Computing Using Ising Accelerators. *Computer* 58, 6 (2025), 83–86. <https://doi.org/10.1109/MC.2025.3544798>
- [11] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyoungwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 43–56. <https://doi.org/10.1109/ISCA52012.2021.00013>
- [12] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. 2020. A modern primer on processing in memory. *arXiv preprint arXiv:2012.03112* (2020).
- [13] S. S. Teja Nibhanupudi, Siddhartha Raman Sundara Raman, and Jaydeep P. Kulkarni. 2021. Phase Transition Material-Assisted Low-Power SRAM Design. *IEEE Transactions on Electron Devices* 68, 5 (2021), 2281–2288. <https://doi.org/10.1109/TED.2021.3067849>
- [14] S. S. Teja Nibhanupudi, Siddhartha Raman Sundara Raman, Mikaël Cassé, Louis Hutin, and Jaydeep P. Kulkarni. 2021. Ultra-Low-Voltage UTBB-SOI-Based, Pseudo-Static Storage Circuits for Cryogenic CMOS Applications. *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* 7, 2 (2021), 201–208. <https://doi.org/10.1109/JXCDC.2021.3130839>
- [15] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287* (2020).
- [16] Yikan Qiu, Yufei Ma, Wentao Zhao, Meng Wu, Le Ye, and Ru Huang. 2022. DCIM-GCN: Digital Computing-in-Memory to Efficiently Accelerate Graph Convolutional Networks. In *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.
- [17] Siddhartha Raman Sundara Raman. 2024. A Review on Non-Volatile and Volatile Emerging Memory Technologies. In *Computer Memory and Data Storage*, Azam Seyedi (Ed.). IntechOpen, Rijeka, Chapter 3. <https://doi.org/10.5772/intechopen.110617>
- [18] Siddhartha Raman Sundara Raman. 2026. *Compute in eDRAM using indium gallium zinc oxide transistors*. Ph.D. dissertation. The University of Texas at Austin. <https://repositories.lib.utexas.edu/items/4dbc7f92-c062-4cb8-b07b-ed29761b9704> Available: <https://repositories.lib.utexas.edu/items/4dbc7f92-c062-4cb8-b07b-ed29761b9704>
- [19] Siddhartha Raman Sundara Raman. 2026. Emerging memory technologies at room/cryogenic temperature. *arXiv:2605.21912 [cs.AR]* <https://arxiv.org/abs/2605.21912>
- [20] Siddhartha Raman Sundara Raman, Lizy John, and Jaydeep P. Kulkarni. 2025. SPARK: Sparsity Aware, Low Area, Energy-Efficient, Near-memory Architecture for Accelerating Linear Programming Problems. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 99–112. <https://doi.org/10.1109/HPCA61900.2025.00019>
- [21] Siddhartha Raman Sundara Raman, Lizy K John, and Jaydeep P. Kulkarni. 2026. A comprehensive study on ILP acceleration accounting for sparsity, area, energy, data movement using near-memory architecture. *arXiv:2605.17158 [cs.AR]* <https://arxiv.org/abs/2605.17158>
- [22] Siddhartha Raman Sundara Raman, Lizy K. John, and Jaydeep P. Kulkarni. 2026. A detailed algorithmic study on a reuse-aware, near memory, all-digital Ising machine. *arXiv:2605.12959 [cs.AR]* <https://arxiv.org/abs/2605.12959>
- [23] Siddhartha Raman Sundara Raman and Jaydeep P. Kulkarni. 2026. ABI: A tightly integrated, unified, sparsity-aware, reconfigurable, compute near-register file/cache GPU architecture with light-weight softmax for deep learning, linear algebra, and Ising compute. *arXiv:2602.14262 [cs.AR]* <https://arxiv.org/abs/2602.14262>

- [24] Siddhartha Raman Sundara Raman, Siyuan Ma, and Lizy Kurian John. 2026. A comparative study on power delivery aspects of compute-in/near-memory approaches using DRAM. *arXiv:2604.04773 [cs.AR]* <https://arxiv.org/abs/2604.04773>
- [25] Siddhartha Raman Sundara Raman, S. S. Teja Nibhanupudi, Atanu K. Saha, Sumeet Gupta, and Jaydeep P. Kulkarni. 2021. Threshold Selector and Capacitive Coupled Assist Techniques for Write Voltage Reduction in Metal–Ferroelectric–Metal Field-Effect Transistor. *IEEE Transactions on Electron Devices* 68, 12 (2021), 6132–6138. <https://doi.org/10.1109/TED.2021.3121348>
- [26] Siddhartha Raman Sundara Raman, Feng Wen, Ravi Pillarisetty, Vivek De, and Jaydeep P. Kulkarni. 2021. High Noise Margin, Digital Logic Design Using Josephson Junction Field-Effect Transistors for Cryogenic Computing. *IEEE Transactions on Applied Superconductivity* 31, 5 (2021), 1–5. <https://doi.org/10.1109/TASC.2021.3054347>
- [27] Siddhartha Raman Sundara Raman, Shanshan Xie, and Jaydeep P. Kulkarni. 2022. IGZO CIM: Enabling In-Memory Computations Using Multilevel Capacitorless Indium–Gallium–Zinc–Oxide-Based Embedded DRAM Technology. *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* 8, 1 (2022), 35–43.
- [28] Siddhartha Raman Sundara Raman, Shanshan Xie, and Jaydeep P. Kulkarni. 2021. Compute-in-eDRAM with Backend Integrated Indium Gallium Zinc Oxide Transistors. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. <https://doi.org/10.1109/ISCAS51556.2021.9401798>
- [29] Rishov Sarkar, Stefan Abi-Karam, Yuqi He, Lakshmi Sathidevi, and Cong Hao. 2023. FlowGNN: A Dataflow Architecture for Real-Time Workload-Agnostic Graph Neural Network Inference. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1099–1112. <https://doi.org/10.1109/HPCA56546.2023.10071015>
- [30] James E Stine, Ivan Castellanos, Michael Wood, Jeff Henson, Fred Love, W Rhett Davis, Paul D Franzon, Michael Bucher, Sunil Basavarajaiah, Julie Oh, et al. 2007. FreePDK: An open-source variation-aware design kit. In *2007 IEEE international conference on Microelectronic Systems Education (MSE'07)*. IEEE, 173–174.
- [31] Siddhartha Raman Sundara Raman, Lizy John, and Jaydeep P. Kulkarni. 2024. NEM-GNN: DAC/ADC-less, Scalable, Reconfigurable, Graph and Sparsity-Aware Near-Memory Accelerator for Graph Neural Networks. *ACM Trans. Archit. Code Optim.* 21, 2, Article 39 (May 2024), 26 pages. <https://doi.org/10.1145/3652607>
- [32] Siddhartha Raman Sundara Raman, Lizy K. John, and Jaydeep P. Kulkarni. 2024. SACHI: A Stationarity-Aware, All-Digital, Near-Memory, Ising Architecture. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 719–731. <https://doi.org/10.1109/HPCA57654.2024.00061>
- [33] Siddhartha Raman Sundara Raman, S. S. Teja Nibhanupudi, and Jaydeep P. Kulkarni. 2022. Enabling In-Memory Computations in Non-Volatile SRAM Designs. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 12, 2 (2022), 557–568. <https://doi.org/10.1109/JETCAS.2022.3174148>
- [34] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [35] Hongwei Wang, Jialin Wang, Jia Wang, Miao Zhao, Weinan Zhang, Fuzheng Zhang, Wenjie Li, Xing Xie, and Minyi Guo. 2019. Learning graph representation with generative adversarial nets. *IEEE Transactions on Knowledge and Data Engineering* 33, 8 (2019), 3090–3103.
- [36] Shanshan Xie, Siddhartha Raman Sundara Raman, Can Ni, Meizhi Wang, Mengtian Yang, and Jaydeep P. Kulkarni. 2022. Ising-CIM: A Reconfigurable and Scalable Compute Within Memory Analog Ising Accelerator for Solving Combinatorial Optimization Problems. *IEEE Journal of Solid-State Circuits* (2022), 1–13. <https://doi.org/10.1109/JSSC.2022.3176610>
- [37] Xinfeng Xie, Zheng Liang, Peng Gu, Abanti Basak, Lei Deng, Ling Liang, Xing Hu, and Yuan Xie. 2021. Spacea: Sparse matrix vector multiplication on processing-in-memory accelerator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 570–583.
- [38] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. 2020. Hygcn: A gcn accelerator with hybrid architecture. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 15–29.
- [39] Tao Yang, Dongyue Li, Yibo Han, Yilong Zhao, Fangxin Liu, Xiaoyao Liang, Zhezhi He, and Li Jiang. 2021. PIMGCN: A ReRAM-Based PIM Design for Graph Convolutional Network Acceleration. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 583–588. <https://doi.org/10.1109/DAC18074.2021.9586231>